



ChemSolve Interim Report

**DT857
BSc in Computer Science(infrastructure)**

Yuriy Kizyuk

C19489214

Robert Ross

School of Computer Science
Technological University, Dublin

5/1/2023

Abstract

The objective of this project is to show how AI's can be incorporated into a chemistry mobile app, to help people who love chemistry but aren't so good at wrapping their heads around some of the concepts in chemistry. I'm hoping that this app will make chemistry more easier and enjoyable while also helping any experts in chemistry speed up their workload, and since it's a mobile app it will be available to anyone with a mobile device.

Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

_Yuriy Kizyuk_____

Yuriy Kizyuk

15/1/2023

Acknowledgements

I would like to thank my supervisor Robert Ross for his patience and advice, he helped me keep on track and pin point exactly what I wanted this project to be about and also helped me along the way with any technologies or software and even explaining the parts of some of the report I didn't understand, without his help and constant push forward this project may have not turned out the way it is.

Table of Contents

1	Introduction	7
1.1	Project Background	7
1.2	Project Description	8
1.3	Project Aims and Objectives	8
1.4	Project Scope	10
1.4.1	Machine-learning model	11
1.5	Thesis Roadmap	11
2	Literature Review	13
2.1	Introduction	13
2.2	Alternative Existing Solutions	13
2.3	Technologies Researched	15
2.3.1	Machine-learning models	15
2.3.2	Libraries, tools and software's	22
2.3.2.1	Cheminformatic tools	22
2.3.2.2	Software and Applications for Development	24
2.3.2.3	Software for 3D models and Visualization	25
2.3.2.4	Software for Application Development	26
2.3.2.5	Web Frameworks	27
2.3.2.6	Libraries for Model development	27
2.3.2.6	Other useful libraries	28
2.4	Other Relevant Research	28
2.5	Existing Final Year Projects	30
2.6	Conclusions	31
3	Experiment Design	32
3.1	Introduction	32
3.2	Software Methodology	32
3.3	Overview of System	33
3.4	Software test plan	34
3.5	Front-end	34
3.6	Middle-Tier	35
3.7	Back-End	36
3.8	Conclusions	36
4	Experiment Development	37
4.1	Introduction	37
4.2	Software Development	37

4.2.1 Project Set-up	37
4.2.2 Generating Training Data	40
4.2.3 Machine learning Model	46
4.3 Front-End Development.....	49
4.4 Middle-Tier Development.....	53
4.5 Back-End Development.....	54
4.6. Conclusions	57
5. Testing and Evaluation	58
5.1. Introduction	58
5.2. System Testing	58
5.2.1 Model accuracy	58
5.2.2 Black Box testing	61
5.3 System Evaluation	62
5.4 Questionnaires and Interviews Evaluation	62
5.4. Conclusions	63
6. Conclusions and Future Work.....	64
6.1. Introduction	64
6.2. Conclusions	64
6.3. Future Work	65
Bibliography	67

1 Introduction

Over the past decade the growing number of AI implementations into our daily lives has rapidly increased, from self-driving cars to virtual assistants and even now chatGPT just in January 2023 has over 100 million users shows us exactly how much AI's have been steadily integrated into our lives. This is mainly thanks to the rapid advancements in AI fields such as deep learning, generative model AI algorithms and even AI in Robotics which has enabled robots to be more intelligent and autonomous which has led to new applications in manufacturing, healthcare, and transportation. With all these new technologies and advancements in AI it has big potential to revolutionize many industries and change the way we live and work while also creating endless opportunities for innovation.

One field that AI can have a significant impact is in chemistry. There have been vast amounts of chemical data generated over the years, this presents both an opportunity and a challenge. The data can be used to make predictions and discover new compounds that can revolutionize industries from pharmaceuticals to materials, but the challenge of this data is that it's often very difficult to analyse and require significant computational resources. Currently on the smartphone market there are many chemistry apps that vary from helping you study to various chemistry games, only a very small handful of these might even have an AI helping them such as KingDraw which uses AI for predicting compound properties. The one thing that I couldn't find is any app that can use AI's to give us more information about molecules with the use of real life imagery.

The project idea is to use these vast amounts of data and make a model that would be able to utilize this data to predict drawn hydrocarbons and show us the information of that hydrocarbon. To facilitate this and make it more available for various users to use, the model will be deployed as an API server that will be interacting with an android app, which will allow users that don't have access to a computer or laptop be able to use the model to its full capability from their own phones.

The project requires some infrastructure to host the application as the model will be running from a server and we will need to be able to process the user data in a safe manner which means that this project is going to be a full stack development with both server-side and client-side development.

1.1 Project Background

The inspiration for this project came from my own experience studying chemistry, I used to really enjoy learning about different atoms and molecules and the bonds they share and lots of different aspects of chemistry, what made more difficult to study the subject was the various maths modules, the maths was do-able for small compounds and reactions but when you got the to the larger reactions and compounds it can get quite difficult and very hard to find where you went wrong, I especially found it hard when we had to convert grams to moles and vice versa or find the moles of a certain atom in a chemical compound, moles itself is hard measurement to get your head around when your new to chemistry as the definition of a mole is the unit of measurement in the international system of units for amount of substance this can get complicated when you don't know if your trying to look for the mole of an atom or the grams of that atom, there are also so many formulas and maths rules you have to follow for chemistry that it's easy to forget a few steps, as such I wish I had an app that could of helped me identify these large structures and solve these complicated equations related to them and also help me easily to find names of these compounds which otherwise could take a long time so figure out.

One of the aims of this project is to be able to speed up the workflow for chemists, even if they know how to do the maths accurately there is still a lot of calculating and maths you have to do which will consume time, whilst you could just have a machine do it for you to speed up the process.

1.2 Project Description

This Chemsolve application is an app that will be running on android, and through this app the user will be able to take pictures of drawn or digital images of hydrocarbons and scan them to be sent to the server. The server will be hosting a machine learning model with an interactive shell that can be used as an API to be able to parse various images of hydrocarbons. The model will use these images to predict what molecule it sees and turn them into a SMILES strings, this is effectively one of the ways how you can deploy a machine learning model to take inputs.

When user opens app, they are greeted with a scan button which will let them scan an image and parse it to the server containing the machine learning model.

Once the prediction is done and the SMILES string is obtained, we can get data from this string, we can find out the chemical formula, the grams of the molecule, the inChi name, the IUPAC name and we can even get file that gives us the 3d coordinates of each atom and bond.

When this data is all converted it is sent back to the user's device, the app then uses all this data to display information about the compound to the user.

The way the data is displayed is that first the app uses the file with coordinates for the molecule to build a 3d structure of the molecule in Unity and display it for the users in a scene. The structure will have triple and double bonds while also showing each atom and individual bond, the user may also feel free to swipe on the screen where the molecule is located to rotate it.

The other information such as chemical formula and chemical name will be displayed alongside the 3d structures in their own little panels which will display all the data.

To send and receive data the server and app will be linked through tcp-udp and port forwarding, where the app will send http requests to the server to send images and receive back data. The App will be hosted on OpenVPN which must be active on both client side and user side for the app to be able to send any data over any network.

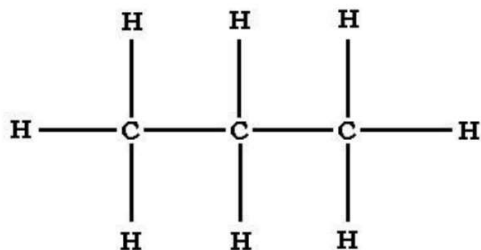
To make the 3d structures, Unity has a custom script made for an empty game object, that will use custom made prefabs for the atoms and bonds to make the 3d structure from the 3d coordinates file it receives.

1.3 Project Aims and Objectives

The overall aim of this project is to be able to utilize this machine learning model to its full capacity using a mobile device. This is effectively done by sending images to the model and manipulating the prediction of those images so that it could display to us all the data about the compound in the image and visualise it for us on our own mobile devices.

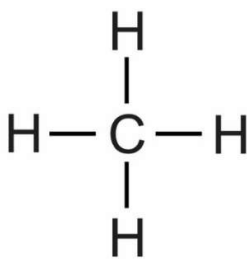
1. The overall objective of this application is to speed up processes in chemistry using machine learning for chemists and students, while it also might have the added-on effect of helping students

and chemists learn faster it will also just generally speed up different chemical processes for the user such as identifying a large hydrocarbon all from a click of button on your phone. A hydrocarbon itself is an organic compound which is made up of just hydrogen and carbon atoms, some of these more complicated structures might even contain cyclic carbon rings and multiple double and triple bonds. Examples of some popular hydrocarbons include methane and propane.



Propane

Figure 1 propane molecule



METHANE

Figure 2 methane molecule

Now to show a more complicated structure.

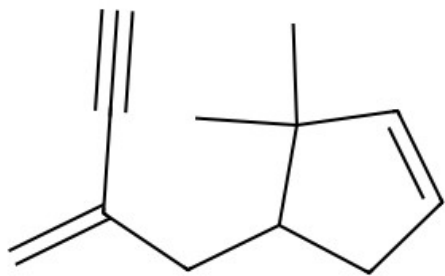


Figure 3 structure for SMILES (C#CC(=C)CC1CC=CC1(C)C)

This is structure taken from our training data represents the molecule name of 3,7-dimethylocta-1,6-diene or 3-methyl-1-phenyl-2,6,10-trimethylcyclododecane or even 4-ethylidene-1-methylcyclohexene, the reason for this is because the name can change depending on how you interpret the double and triples bonds and where they are located. As you can see the structure

doesn't even have any atoms labelled on it, so the model needs to be able to recognise that the structure is hydrocarbon which means it can only have carbon and hydrogen atoms in its structure.

Here are some more words I use throughout the project: molecule which is a group of 2 or more atoms so most hydrocarbons are molecules, A compound similar to molecules is a substance that's made up of 2 or more different chemical elements combined in a fixed ratio examples of this would be sugar, water, salt etc, it has to be fixed ratio because for example water is made up off hydrogen and oxygen, now water can only have 2 hydrogens for every oxygen that's why it's H₂O, so it has to be in a fixed ratio of 2:1 as 3:1 wouldn't be water anymore.

2. To complete the overall objective we have another objective which is making the machine learning model. This model must be able to parse in images of hand drawn or digitally drawn hydrocarbons and give us back accurate predictions of SMILES strings regardless of the background noise of the image. Meaning that even if the image has page lines going through the structure or other background artifacts in the image the model should still be able to focus solely on the drawn structure and give an accurate prediction in the form of a SMILES string.

3. Another objective is to make a Unity project and export as an android project. First we need to make the unity project to be able to set up a scene with lighting and a camera pointed at an empty game object, this empty game object will be assigned a script that will build the 3d structure of the image taken. Then once that's done, we need to export it as an android project so that it could be used as an android instance.

4. To add to the Unity project objective we also need an android project objective, this objective consists of us building an android app to hold the unity project scene and script and also to be able to scan images to send and receive data from the server.

5. The machine learning model must be hosted on a server as an API so that it can be accessed relatively easily without any hassle.

6. And the server must also be hosted on a domain so that it can be accessed over any network or any mobile network without any trouble.

1.4 Project Scope

There are few areas of development need for this project, firstly we need 2 machine learning models, one for detecting the hydrocarbon the user was wrote and second being a prediction model for the products of reactants, we also need functions to balance out the equations between reactants and products and functions to be able to convert grams to moles and moles to grams and lastly we also need to fit all of that into a mobile app.

1.4.1 Machine-learning model

The challenge of recognising chemical structures is supervised learning problem, most likely this model will be a convolutional neural network or CNN as they are most often used when trying to classify images. The machine learning model will be trained and built using TensorFlow and NumPy libraries to do lots of the calculations and testing for us such as computing the levenshtein distance between two images, EM scores or exact match scores and blue scores for the inputs and outputs of the model. This will help us see how well trained our model is and how accurate its predictions are and minimizing error between the predicted NN label and the reference label.

I'm planning to train it on multiple sets of data such as one being clean images of the structure and others being rotate and distorted and also hand drawn images with lots of background noise. This will help model recognise the structure regardless of if its digital or hand drawn and distinguish it between all the background noise. The model itself wont be doing the error checking to see if the scanned structure is a real structure because since it will trained to predict a label for the structure regardless of the background it will always make a SMILES label even there isn't a structure in the image, the way we can error test is using chemical python libraries such as RDKit which not only helps render images but can also tell us the chemical formula of a SMILES string for reference this is what one looks like C#CC(=C)CC1CC=CC1(C)C, and if the smiles string is an invalid structure the RDKit will let us know it's not a real structure which in turn error checks what the ML model is outputting.

1.5 Thesis Roadmap

This section shows an overview of what the upcoming chapters are about.

2 Literature Review

This chapter is all about investigating any existing solutions to the problem that this project is trying to solve, it explains how all the different libraries and model structures in these existing solutions can be used and combined into this application using previously researched technology.

3 Experimental Design

The purpose of this chapter is to show the development approach to be taken during the project. This is done by explaining the project methodology and systems and showing how the system architecture all fits together.

4 Experimental Development

This section goes into detail how each component was designed and why I used the libraries and software that I did and how they effected the decisions in the project.

5 Testing and evaluation

This chapter aims to provide how I went about the testing the different systems and the model's accuracy.

6 Conclusions and Future Work

Within this chapter we discuss the final conclusions we came to with this project and any future developments I may have in mind.

2. Literature Review

2.1 Introduction

As can be seen from the above section we can tell there will be quite a lot of research needed to be put into this project, from finding existing libraries, projects and examples for the machine learning model which is a whole complex system in itself, to also being able to deploy the machine learning model so the user can interact with it which will include both server side and client side development, we also need to find a way so that our devices can access the model over any network and lastly we have to find out how to develop an android app to connect to our server and send receive requests from it and as well as being able to include our other functionality such as scanning and sending an image to displaying the scanned structure as a 3d-model. So we have quite a bit of research to be done which will be dived into below

2.2 Alternative Existing Solutions

I first went and looked for libraries and models that already exist to my current problem and the results I came about all turned out to be very interesting as the problem for classifying chemical structures and images has been one problem that has been worked on throughout the decades, I have found multiple libraries and models that have been created in various years and all using the various tools made in those years.

ChemInk

The first interesting solution I came across was chemink by [Xuguodong](#)[1], this was handwritten recognition software for chemistry related formulas, it was able to balance chemical equations and generate shape of layered chemical structure for all isomers of an n-C alkane, a n-alkane is a saturated hydrocarbon which means they consist of hydrogen and carbon with single bonds only. This software was also able to render stick and-ball models for these isomers.

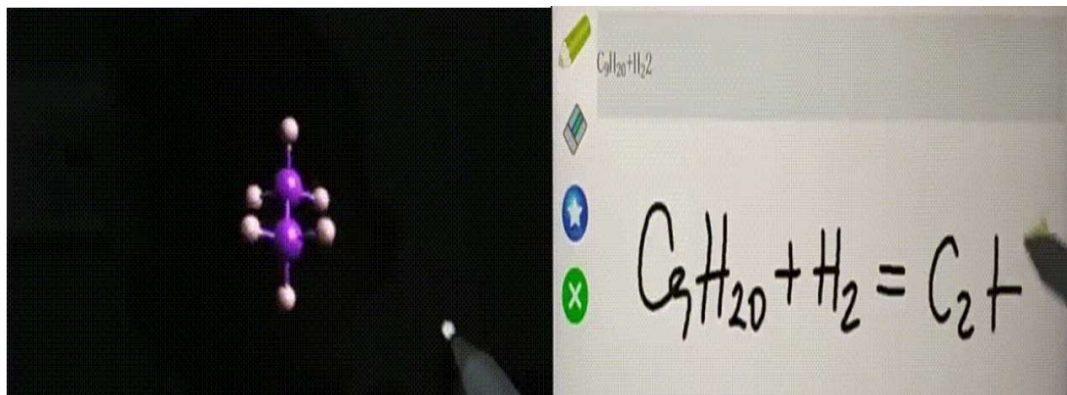


Figure 4 interacting with ChemInk

The system uses yolov3-tiny, which is object detection model for cutting out characters, to cut out the consecutive pen strokes, a CNN or convolutional neural network is used with densenet structure to reclassify the characters in the detection box. Then the bitwise weighting method is used to further filter reasonable results from the linguistic level. It also uses a OpenCV inference object detection network, fdeep keras model, fdeep is a library for using keras models in C++, the model is used to find inference of single characters, and the basic 3D elements were provided by GLU. It was hard deciphering what this project was exactly doing because most of it was in Chinese, but it did introduce me to some interesting ideas and many different technologies that I can use for the detection such yolov3, keras and other libraries such as openbale and libxml2. One common theme I found throughout these structure detection models is that they mostly use CNN's model types.

MolAR

The next project I looked at was MolAR by the [mtzgroup\[2\]](#), MolAR is an open-source mobile application for visualizing molecules in AR which can be directly visualized from their hand-drawn structures or you can visualize any molecule or protein directly from its name or a protein data bank ID which lets you scan source of the protein e.g. to get caffeine you can scan coffee, it also computes chemical properties in real time via quantum chemistry cloud computing. This project is interesting because its doing something similar to what I want with augmented reality, but it also uses a very large databank to get all the different proteins, molecules and compounds which I neither have the time nor resources to replicate this as such this isn't the best example of how to go about it for my project but it does give us some useful insight on how it can be used in my project, as my project can solely focus on hydrocarbons which mainly made of carbon and hydrogen atoms. This eliminates a lot of complexity while also showing how capable AI's are at recognising chemical structures and that with a lot more time and resources we could potential make an AI that can very accurately predict any chemical compound or substance and even discover the possibilities for new ones.

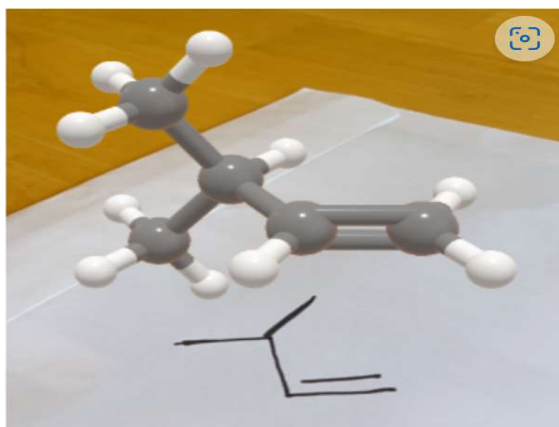


Figure 5 MolAR app on drawing

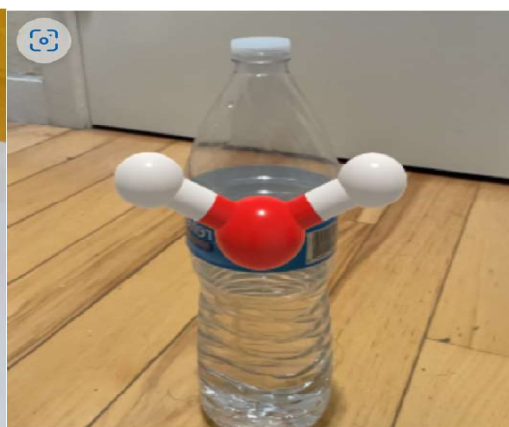


Figure 6 MolAR on real life object

This application has 2 main components one being the iOS app and the other web server that mediates between the app and cloud services, the web server helps the app use all these different

features as it changes between different environments such as Mathpix to use the chemical structure recognition, Amazon Rekognition is used for object recognition feature, to use quantum computation you need a TeraChem account. While this app had many interesting features I was more interested as to how they detected hand drawn molecule, for the detection system they used another model mtzgroup created called ChemPixCH which was hand drawn hydrocarbon recognition system that used generated data and neural network model that was modified from the Im2Latex network by [Guillaume Genthial](#) [3]. This system uses data generated by building datasets for training, validation and test sets for clean RDKit images, synthetic RDKit images and hand drawn images and a tensor flow neural network model to train the data on. This is another CNN model and also uses the cheminformatics tool RDKit which I found out can be very useful for our project in many different aspects, it also shows how they deployed their model and also all the other resources they used for the app such Amazon Rekognition, this gives me the idea to also use other online tools to make the functionality of my app easier.

DECIMER

The final open-source project I looked at for hand drawn recognition is DECIMER by [Kohulan](#) [4], now this application has 2 versions one just being an updated version of the other but will start with the first DECIMER V1 images to SMILES. This also first generates the images using SDF or SMILES files to generate the text data, the text data is DeepSMILES strings as generated using `Deepsmls_Encoder.py` to turn SMILES to DeepSMILES. This data is then used to train the TensorFlow model and then the model is evaluated by scanning an image to predict its SMILES chemical formula.

The other DECIMER V2 image transformer differs from the previous software as the data for this project was trained on TPU instead of GPU and also now uses EfficientNet-V2 for image feature extraction and a new transformer model for predicting the SMILES which is still made using. So the main changes here is how the image is extracted and how the model is trained as now the datasets are converted to TFRecords which is a binary file system that the TPUs can read.

2.3 Technologies Researched

2.3.1 Machine-learning models

We discovered many different why of machine learning model infrastructures, here I'm going to list the most the popular and useful ones to tackle this problem, again this problem is one for supervised learning so most of these will supervised learning models.

CNN's

The first and most popular being CNN's or convolutional Neural Networks, these are networks that consist of several layers of convolutions and pooling operations, followed by one or more fully connected layers.

Benefits:

These CNNs are particularly well-suited for tasks that involve extracting features from images. These can be especially useful for chemical structure extraction as they can learn complex hierarchal representations of features from raw image data and without requiring any hand-crafted feature engineering.

Since they are most widely used for image classification, they always have state-of-the-art technologies for predicting performance on many datasets.

CNN's can also be trained on very large-scale datasets meaning they could take up a lot of time to train, one way this process is increased is by the use of GPU acceleration, and since my machine contains an Nvidia 3080 GPU the process to learn highly accurate models in a reasonable a reasonable amount of time really boosts the benefit and usability of this model as we can make any changes to the model or data and get a test version up and running very fast.

Drawbacks:

Since the CNN's can predict with such high accuracy because of its large-scale datasets this also means we need a very large amount of supervised training data to achieve high accuracy.

Now while the GPU's can exponentially increase how fast a model can be trained and its computational power this also means you need an equally powerful GPU to match this computational expense, which might not be available to everyone which could put people off using this infrastructure.

Since CNN's have improved so much over time and so much has been added it can be difficult to interpret the learning features in a CNN, making it challenging to understand how the model is making its predictions.

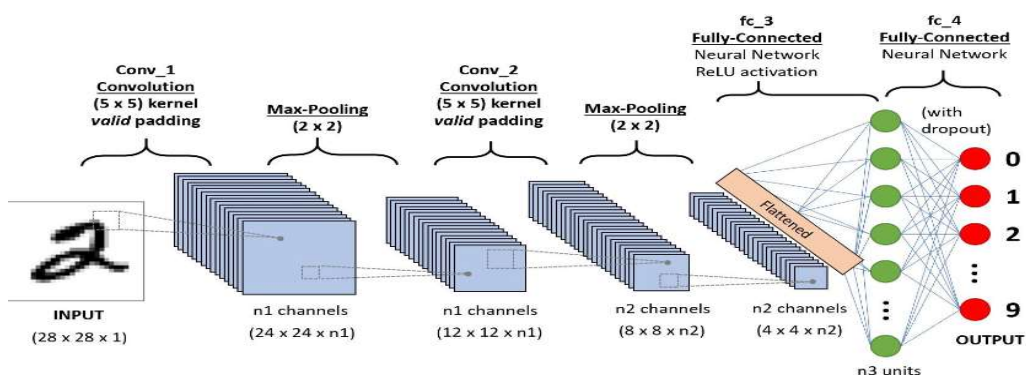


Figure 5 CNN model infrastructure

SVM's

The next model type is SVMs or support vector machines, these are also a popular choice but for binary classification tasks, these work by finding the hyperplane that best separates the data into two classes, these types of models are especially effective when the number of features is large compared to the number of samples.

Benefits:

These are particularly useful because unlike CNN's they are effective at handling high-dimensional data with small training datasets, this makes it handy if we can't find much of the data that we require for the model which in turn decreases training time, testing time and how long it would take to label each training set.

SVM's are easier to read as they work by finding a decision boundary that maximizes the margin between the two classes, this removes a lot of the complexity that the CNN has and give an insight into how the model is making its predictions.

Drawbacks:

SVM's can also be quite computationally expensive when the number of features or training samples are large, which could mean that a pure SVM model could be quite inefficient in classifying features in chemical structures as the structures of hydrocarbons can be quite confusing and require the knowledge of many features in them.

Since they are a bit less more complex than CNN that means that it may not perform as well as other models, such as CNN's, on more complex classification task, which could be a problem for chemical structures.

SVM's can also be very sensitive to the choice of kernel function and its associated parameters which could cause us to run into more problems later on.

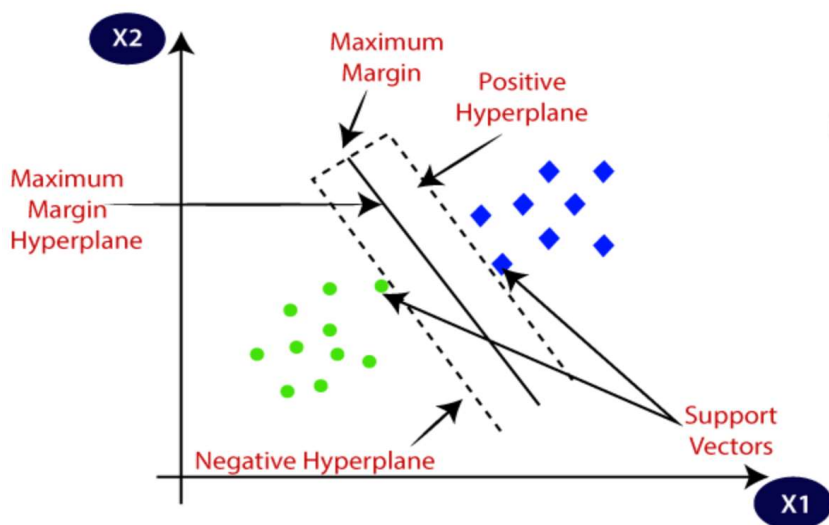


Figure 6 Graph to show how SVM algorithm works.

Decision Trees & Random Forests

Decision tree models are an also popular classification choice, these are simple and effective models for image classification tasks, the way they work is that they recursively partition the data into

smaller subsets based on the features of the images, then at each node the model chooses the feature that provides the most information gain and separates the data into different classes. Now the main problem with these is that its easily prone to overfitting, this is where random forests come in which can help mitigate overfitting by pruning and ensemble methods.

Now Random Forests are an ensemble learning method that can be used for classification and regression tasks, these work by combining multiple decision trees to improve accuracy and reduce overfitting. They are basically randomly selecting subsets of features and data samples and build decision trees on each subset, this way it learns the characteristics that differentiate one class from another. So once this model is trained it can classify new images by taking the majority vote of the different decision trees.

Benefits:

Decision trees themselves are easy enough to interpret which means they can provide useful insights into how the model goes about the process of its decision making, making it easier to modify to manipulate to do what you want.

The model can handle both categorical and numerical datasets.

Decision trees can also handle any missing values in the data and as such can be very good with images and datasets that contain lots of noise.

Random forest themselves even though they are made up of multiple decisions trees, they are actually relatively fast to train, which makes them useful for large training sets.

They can handle non-linear data and as such it helps them capture complex interactions between features.

As mentioned before random forests also mitigate the overfitting of decision trees and also provide estimates of feature importance, which can be useful for feature selection.

Drawbacks:

The main drawback of decision trees is being that they are prone to overfitting if the tree is too deep or if the algorithm isn't properly regularized, and as stated we use random forests to avoid this but while this is good it's also a negative as a whole because dealing with one decision tree is fine but the complexity begins to increase when you have make a bunch of them for a random forest.

The decision tree could perform badly on very imbalanced datasets, which means we would have to filter our dataset for it to balance which can cause problems in itself.

Random forests can also be very computationally heavy, especially when the number of tress in ensemble is large, which would be the case for our use case as we need to classify very complex structures.

Random forests can also succumb to overfitting if there isn't a large enough data supply or if the model itself is too complex.

But also Random forests are just not as effective as other models such as CNN's for image classification tasks with complex structures.

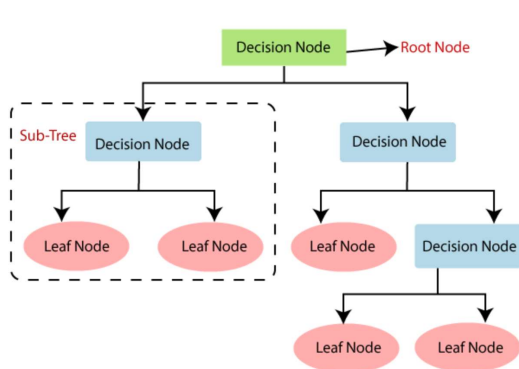


Figure 7 Decision Tree node diagram

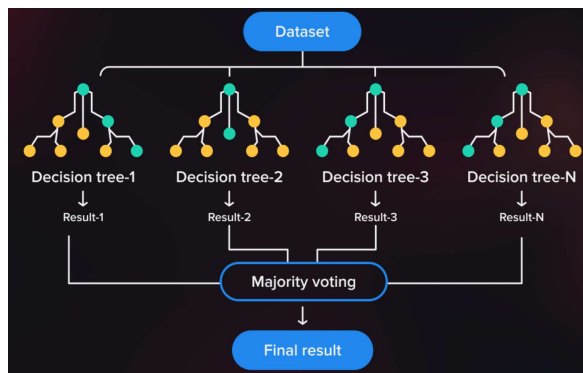


Figure 8 Random Forests made up decision trees

Naïve Bayes

The naïve bayes model is probabilistic type model, meaning that it actually predicts depending on probability and can be used for classification tasks. This is done by the model being trained on a set of labelled images to learn the probabilities of each pixel being associated with a particular class and as such once the model as been trained it can calculate the probability of each class given the pixel values of the image and selecting the class with the highest probability.

Benefits:

Naïve bayes model actually has a simple and effective probability algorithm for classification tasks that can be easy to understand implement.

This type of model can also handle high-dimensional data well and works well with small training sets making it well-suited for image classification.

The model can also work with imbalanced datasets and has relatively quick training time.

Drawbacks:

The Naïve bayes model can assume that all features are independent, which may not hold true to the image it's trying to classify, in our case this is a problem as in the chemical structure it has many features depending on each other.

The model may also not be as effective as CNNs for image classification with our complex image structures, it also may perform poorly if there are correlations between features.

Naïve bayes is also sensitive to the presence or irrelevant features so things such as background noise which can really throw off the model.

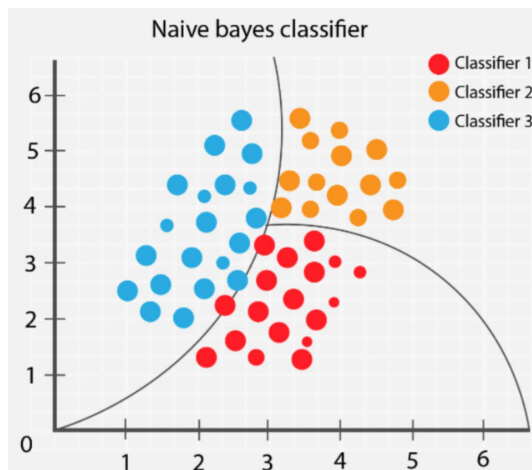


Figure 9 How Naive Bayes algorithm works

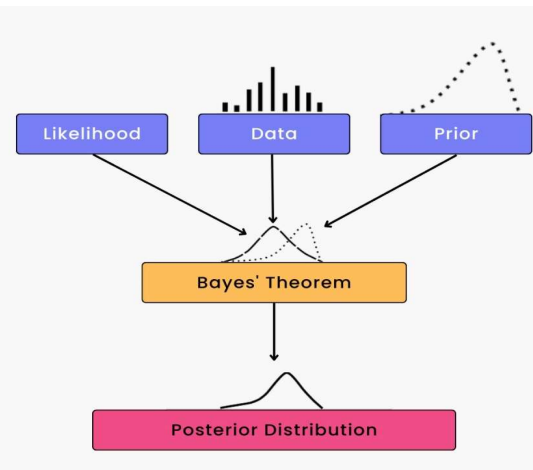


Figure 10 How Naive Bayes Model makes predictions

KNN's

KNN's or K-Nearest Neighbors is a simple yet effective classification algorithm which is non-parametric machine learning method, it uses its classification algorithm to find the k closest data points to a test point and assigns the class label that is most common among those k points. So basically once the model is trained on a set of labeled images it can start to learn the characteristics that differentiate one class from another and as such it can classify new images by finding the K nearest labeled images in the training set and taking the majority vote of their classes.

Benefits:

The model is both non-parametric and simple so the algorithm should be easy to implement and understand.

It can also handle very complex high-dimensional data which is what makes it useful for image classification tasks where each image has a large number of pixels.

KNN can also work very well for small datasets and for datasets with imbalanced class distributions.

Drawbacks:

The KNN model can also be computationally expensive, especially for large datasets, which is especially problematic for anybody that doesn't have an up to standard PC or laptop.

The model requires very careful selection of the number of neighbours (K) and the distance metric.

KNN may not be as effective as other models, such as CNNs, for image classification tasks with complex image structures.

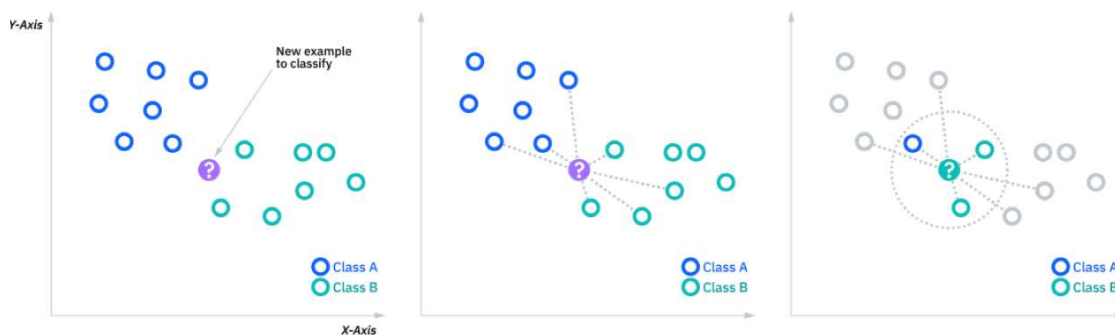


Figure 11 How KNN makes predictions

DNN's

DNN's or Deep Neural Networks are also a class of neural networks that can be used for a variety of machine learning tasks, including image classification, the only difference is that CNNs, which are specifically designed for image classification, DNNs can be used for a wider range of tasks. Like CNN's, DNNs also have a series of layers to learn increasingly abstract representations of the input data and can be trained using various optimization techniques such as backpropagation.

Benefits:

They have really high accuracy at classifying images especially when dealing with complex images, as they learn end-to-end from raw data to output labels without the need for feature engineering or manual feature extraction.

Pre-trained DNNs can be fine-tuned for a new task, reducing the need for a large amount of labelled data, which can be useful if we ever decide we want to classify other molecules and compounds other than hydrocarbons.

DNN's can also easily scale to large amounts of datasets high-dimensional feature spaces.

Drawbacks:

DNNs require a large amount of labelled data for training and required high Computational complexity and significant processing power to achieve good performance, which can be a challenge for resource-constrained systems.

DNN's can also suffer from easily overfit to the training data, which can result in poor generalization performance on new, unseen data.

DNN's use complex systems and layers which can make it difficult to interpret how they arrive at their classification decisions.

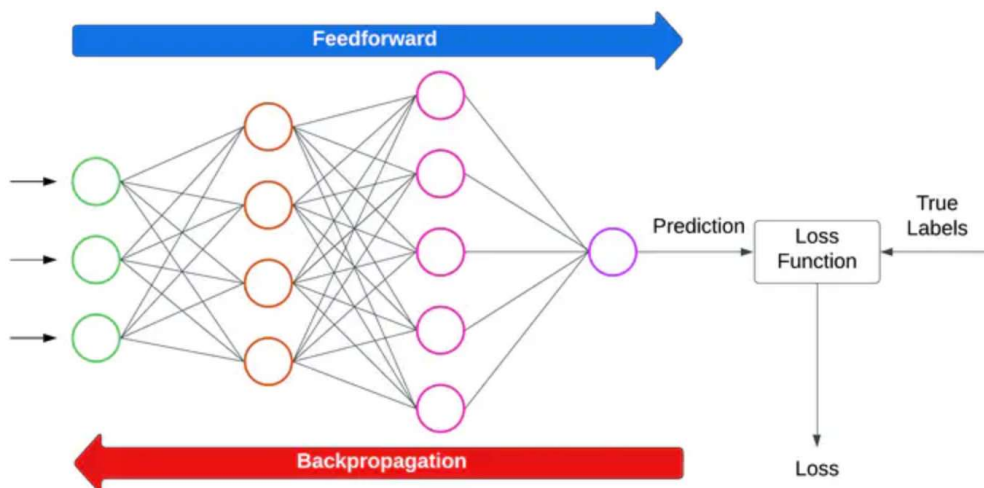


Figure 12 DNN infrastructure

This image shows how DNN's work, which are very similar to CNN's infrastructure with all the layers and predictions.

2.3.2 Libraries, tools and software's

While the machine learning models can help us do the maths as to how to classify each image we also need libraries and tools that can help with other aspects such translating the smiles, rendering the images, hosting servers and building android apps, here is a list of potential libraries and tools we can use to speed up the process of our project.

2.3.2.1 Cheminformatic tools

-SMILES

Smiles or **simplified molecular-input line-entry system** is used to change a chemical's 3-D or 2-D structure into a string of symbols, these symbols can then be understood by different computer software to give us different information about the structures such as positions of bonds and atoms, chemical formula, the inChi representation and even turn the smiles into an SDF file. This gives a SMILES string lots of versatility once it's been correctly predicted.

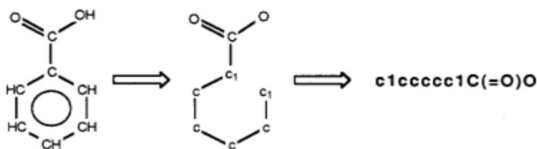


Figure 13 How structure converts to SMILES

This above image shows an example of how the SMILES notation is retrieved from a structure, with the c and C signifying a carbon atom and O as the oxygen atoms, the single bonds are '-' but usually aren't shown, only the double bonds as '=', triple bonds as '#' and quadruple bonds as '\$', it also knows that the c's are each connected to their own hydrogen making a cyclohexane or dioxane as shown by CH's in a cyclic structure, for this smiles just makes the first carbon to have a hydrogen bond a c1 and the last one to have a hydrogen bond also c1, which signifies which carbon atoms are in the cyclic structure.

-DeepSMILES

The problem with smiles is that a lot of the times smiles strings aren't syntactically valid as elements of SMILES syntax must occur in pairs. This is where we get DeepSMILES which like SMILES is also an Simplified Molecular Input Line Entry System notation that is commonly used to represent chemical structures as text strings but not only is it a simplified version of SMILES it also addresses the main reasons for invalid syntax this is done by avoiding unbalanced parentheses by only using close parentheses and so it mitigates the problem of pairing ring closure symbols by using only single symbol at the ring closing location. So, it a distinctive advantage over traditional SMILES as it can represent more complex molecules, the ones which have multiple rings and branches but in more compact and standardized format.

But the downside to this is that is that DeepSMILES may not be as intuitive or human-readable as traditional SMILES for chemists and researchers who are used to working with chemical structures and because it is a simplified version of SMILES, it may not capture all of the features and details of complex molecular structures but since we are only looking for hydrocarbons the usage of DeepSMILES doesn't seem necessary as hydrocarbons only consist of Carbon and Hydrogen atoms and but the representation in DeepSMILES wont be as detailed and could skip information about the bond type. As such I decide to skip looking through SELFIES or BigSMILES notations as SMILES would be the most suitable one anyway.

-OpenBabel

OpenBabel is a chemical toolbox, that is used to speak many languages of chemical data, it is an open-source software package designed for cheminformatics and computational chemistry. This chemical toolbox provides users with a wide range of tools for anyone to be able to search, convert, analyse, and store data from molecular modelling, chemistry, solid state materials, biochemistry or related areas. It also has computing properties such as molecular descriptors and fingerprints, which can help us analyse chemical structures.

Now the reason why we might want to use this the most is because of its support for multiple chemical file formats, including SMILES. OpenBabel is able to read and write SMILES strings and also change SMILES strings to other formats, such as molecular graphics files or database formats. It can also generate several conformations (i.e., different arrangements of atoms and bonds) for a given molecule which we can use later on as coordinates to build 3D structure from later in Unity.

-RDKit

RDKit is also an open-source cheminformatics toolkit which was written in C++ but is also useable in java and python, and like OpenBabel it also is a software package for cheminformatics and computational chemistry that provides a wide range of tools for working with molecular structures and analysing their properties. This toolkit includes a lot of the same tools as OpenBabel such as reading and writing SMILES strings, as well as convert SMILES strings to other formats, such as molecular graphics files and SDF files it also has a collection of standard cheminformatics functionality for molecule I/O, substructure searching, chemical reactions, coordinate generation including 2D and 3D, as well as fingerprinting ect and a high-performance database cartridge for working with molecules using PostgreSQL database.

For the project I think I'm going to use RDKit instead of OpenBabel as I think RDKit has a more user-friendly and intuitive Python API, which makes it easier to integrate with other Python-based libraries and tools for image classification, such as TensorFlow or PyTorch and my machine learning model will most likely be written in python and use libraries such as TensorFlow or PyTorch.

2.3.2.2 Software and Applications for Development

-Anaconda

This is very popular open-source distribution software of the python language for scientific computing, it includes a number of useful libraries and tools for data science, machine learning, and scientific computing and aims to simplify package management and deployment. Some of these libraries include NumPy, Pandas, Matplotlib, and Scikit-learn. We can use this to manage all the libraries, packages and dependencies that will be required by our model, this is done through the control of custom Python environment that you can make in anaconda to keep and track of all the dependencies and packages while also allowing us to install new ones. Anaconda has multiple interfaces and IDE's you can use such as we can use Jupyter Notebook, PyCharm and Spyder which will all allow us to code in python and do different scientific computing, it also has its own CMD and Powershell which we can use to make environments and add different packages and libraries.

-Jupyter Notebook

Jupyter notebook is an open-source web application that can be ran inside Anaconda with one of its pre-set or custom Python environment to be able to access the same libraries, this web app allows you to create and share documents that contain live code, equations, visualizations, and narrative text but we will mostly be using it to visualize and generate our data and train our models on it.

-PyCharm

This is a Python IDE or Integrated Development Environment which is specifically designed for Python programming language, this is also one of the available resource in anaconda that can utilize our Python environment. This software was made by JetBrains and provides developers with a suite of tools for developing, testing, and debugging Python code, so this would also be mainly used to code and test our models.

-Visual Studio

This is another IDE but this time developed by Microsoft and is open-source, this IDE is mainly used for building applications for Windows, Android, iOS, and the web and is also one of the most popular IDEs available. This IDE is actually very impressive as it supports a plethora of programming languages including C++, C#, JavaScript, Python, and many more, and with all these languages it also supports all their corresponding libraries and packages. Another very useful feature is that we can also use custom Python environments in this IDE, so we have any custom-made ones or pre-set Python environments in anaconda they will automatically be detected by Visual Studios for us to use and also giving us free range to manipulate those environments as well. Visual Studios also has the abilities to run servers off it so we can test different server API's on it.

2.3.2.3 Software for 3D models and Visualization

-Unity

Unity is open-source game engine that allows developers to create 2D and 3D games for various platforms, such as mobile devices, computers, gaming consoles, and virtual reality devices. This could be a perfect software to use so we could display our structures in 3D on the mobile device, it uses C++ for scripting so we can make scripts in Visual Studios to make the 3D structures and utilize them in Unity. Unity like most game engines has a wide range of tools for game development, including a visual editor, scripting tools, physics engine and animation tools, meaning the 3D models for the atoms and bonds can be made directly in Unity along with materials, lighting, backgrounds, HDRI's, camera and animations for our 3D structure so that we could display 3D structures but in more spectacular fashion. It also has high compatibility with software's like Android studio which can make it very easy to incorporate Unity into our App.

-Blender

Blender is also open-source 3D creation software and similarly to Unity is also used for making animated films, visual effects, video games, and 3D models and also has the same capabilities to apply scripts and set up scenes for the 3D structures. While I wont be using this specifically to generate the 3D structure because it's a bit more complicated to integrate with Android Studios, it could be useful for making the 3D assets for atoms and bonds and setting up the materials as I have more experience with Blender which makes sense to make the assets in this software and import them into Unity to use later.

-Rajawali

This is also an open-source 3D graphics engine for Android, and similarly to the two above its also used to simplify the process of creating 3D applications for mobile devices. This functions a bit different from Blender and Unity as to use this you need make a Rajawali viewer, which is kind of like the image viewer you can add in the linear layout of the android studio app and can specify where it

will be on the screen and what dimensions it has etc. It also uses multiple rendering techniques such as OpenGL ES 2.0 and 3.0.

But while this is meant to be an easy-to-use engine, I found it extremely tedious to try set up especially with its limited examples and tutorials online as most of them just wouldn't work, it was much easier to set up a scene with lighting, shading, animation and sound effects with Unity and Blender as I think they have much easier and more interactive process of doing this.

2.3.2.4 Software for Application Development

-Android

Android is one of two major platforms that currently dominates the mobile operating system market, with android ranking as number 1 with having a "71.95% market share in Q1 2023" [5], The operating system itself is based on a linux kernel and provides developers with its own developer kit called Android studios and is mostly used for smartphones, tablets, and other mobile devices.

Android supports multitasking and has is very developer friendly so I will be able to multiple apps simultaneously and switch between them seamlessly while also having an easier time with its range of tools and resources for developers. This is important because the app must be able to support to run a Unity instance while also having the capabilities to access the camera and send pictures to our server so there will be many tasks running at the same time.

-IOS

IOS is the other of the two major operating systems on the market, it has a market share of 27.71% [6]. This is a operating system created by Apple for apple devices, its also closed-source, proprietary system that is exclusively available on Apple devices and requires you to ask apple permission to develop apps on Apple. The one big advantage of apple is its uniformity and consistency, since IOS only runs on Apple devices there is a much smaller range of hardware to optimize for, resulting in a more stable and reliable user experience and also App Store is also highly curated, ensuring that apps are of high quality and do not compromise user privacy. Like android apple also has Xcode in which you can develop IOS apps.

The big problem for me is that even though I would like to make this app for both devices to get a better user testing experience, I wouldn't have enough time to make both apps and since IOS requires Apples permission to even test apps on Apple device this would have increased the waiting time significantly. I also own an android device so it would make more sense for me to stick with them.

-Android Studios

Android studio is an IDE used specifically for developing Android applications, this IDE was developed by Google and uses Java language primarily to build its apps but can also work with C. Android studios provides an emulator that will allow us to test our app on different virtual devices with different configurations such as various SDK's and specs, it also has a seamless integration with

Gradle which will make it easier to manage dependencies and build Android apps. So this will be our IDE for building the App going forward because as well as me being familiar with it, it is also very useful to be able to test our app on different environments with its emulator tool.

2.3.2.5 Web Frameworks

-Flask API

Flask is a Python web framework that will allow us to build web applications quickly and easily. It is a very lightweight and simple framework that follows the Model-View-Controller (MVC) architecture pattern, this is perfect for us because we only need a very simple web app for our machine learning model, so this will be very quick and easy to set up, it also as mentioned lightweight which is also very useful as it will not require a lot of system resources to run which will be paired with the ML model which can be very resource heavy so it makes more space for the model to be efficient.

-Fast API

Fast API is also another Python web framework and built upon the Starlette framework which provides a modern, high-performance web framework and is designed to be fast, easy to use, and highly scalable. Not only that but also offers automatic documentation, and automatic data validation, which can help developers build better APIs more efficiently.

But for this one I think I'm actually going to stick with Flask as I have familiarity with using it and also Flask API is a simpler framework than FastAPI and may be a better choice for small to medium-sized projects, and since the ML model isn't going to be in full blown web app, I think it's the better choice as this web app is only small project just so that it could receive and send requests to the phone app and ML model.

2.3.2.6 Libraries for Model development

-TensorFlow

TensorFlow is machine learning library that can help us make DNN's and CNN's, it open-source and was also developed by Google and provides a large variety of tools and API's that we can use to build our models, it's already designed to be scalable and efficient which makes it very suitable for large-scale machine learning tasks.

The reason this library is interesting is because it has a lot of tools for making CNN's and also already has pre-built layers for convolution and pooling, as well as optimization algorithms for training the network and also it has GPU support meaning we can use its GPU support to speed up the training of our models and also increase computation.

-Keras

Keras like TensorFlow is also an open-source neural networks library with a python interface. Keras is really good because it provides so a simple interface to build and train neural networks, you can

also use different backends such as CNTK, TensorFlow, and Theano. It's really able to simplify the process of building and training neural networks which can save a lot of time and be less complex, this is because with Keras you can easily define the structure of your neural network using a few lines of code it's also really modular and flexible making it easy to swap around the different layers and activation functions to build a custom neural network architecture that could suit our image classification model for our project.

-PyTorch

PyTorch is another open-source machine learning library, that's also for Python, but this library is based on the Torch library, which is which is a popular machine learning library written in Lua. The software was also designed to integrate into python really well which means to write and read the code is very simple as it can look very similar to python code making it much easier Python library to learn and to integrate.

2.3.2.6 Other useful libraries

-CudaToolKit

This is a powerful software development toolkit that will be required for our ML model as it will provide us the necessary tools and libraries for developing high-performance computing applications that run on NVIDIA GPUs, meaning this what we use to enable GPU usage for model so that we can train and run it using the GPU instead of our CPU to give it that boost in computational power which will save us a lot of time.

-CuDNN

Now with Cuda toolkit we need to use it with another library called CuDNN or CUDA Deep Neural Network, this what will let us provides high-performance primitives for deep learning applications and is used to accelerate the training and inference of deep neural networks on NVIDIA GPUs. So we will have to use CuDNN and CudaToolKit to make the GPU's available to use for our machine learning model. For this to actually work when we download CuDNN and CudaToolKit into our Python environment we need to make sure that both versions of these libraries matches with our GPU make e.g. For my Nvidia 3080 GPU I need CuDNN version 8.1 matches with CudaToolKit version 11.2, if I have any other versions then my GPU won't work, also note that for these libraries to work you also need to match that with the TensorFlow version, so those versions would only work for TensorFlow version 2.XX.XX.

2.4 Other Relevant Research

I also needed to find a way to make the ML model server be visible over any network for the mobile app to connect to and since my port forwarding for my router isn't working properly due to my WiFi company's negligence, I had to look for other ways to find a solution to this problem, as I didn't want to host the server on a proper domain as it would cost money I had to look at other options.

-DuckDNS

DuckDNS is a free dynamic DNS software (Domain Name System) that is hosted on Amazon VPC, this software allows you to assign a domain name to your home network's public IP address, this is especially useful because if you have a public IP that changes often DuckDNS will solve this by creating a personalized domain name that points to your dynamic IP address, this means that even though your public IP can change frequently it's still assigned to a static DuckDNS address that won't change, so it updates your domain name automatically when your public IP changes.

-Ngrok

Ngrok is a library that will allow us to make secure tunnels from a public endpoint to a locally running web service. So basically this gets around our port forwarding issue by making tunnels between the server and the app and it was pretty easy to set up and get working. Now while this did get over my initial issue of not being able to connect over any network, it only fixed it temporarily, because for the tunnel to work you would have to start a Ngrok session and take its Ngrok URL and replace it in your android mobile app, but the issue with this was that the session only lasts 2 hours before you have to restart and make a new session and also change the Ngrok URL again and rebuild the entire app with the updated URL again for your mobile device. This made the Ngrok tunnels much less useful for overall useability but does provide us with a way for testing and also an alternative way for demoing in case anything goes wrong.

-OpenVPN

OpenVPN is different from Ngrok as it's an open-source VPN or virtual private network, but it provides us with similar functionality as it uses encryption and tunnelling protocols to create a secure connection, allowing users to access a private network remotely, now this will basically allow us to access resources on a private network, such as files, applications, and servers, from anywhere in the world, which basically fixes our problem and on top of that it also has enhanced security by encrypting all traffic that flows through the VPN which won't let any unauthorized interception of any traffic. It also has no session limits or anything like that so it is perfect to use as the standard way to connect our app to the server, the only thing is the VPN must be activated client-side and server-side, so both the mobile device and PC must have the VPN on for it to work.

-OkHttpClient

This is a popular open-source client library for HTTP and HTTPS connections for android and java applications which makes it perfect to use for our project, it has a very simple design to be able to make network requests from the server and then being able to handle the responses from the server in an easy-to-use way.

2.5 Existing Final Year Projects

Project 1

Title: MeetMeHere

Student: Martin Bruveris

Description: A hybrid mobile App for both Android and IOS that allows users to share location in real time, this done by the host being able to set markers on the map and the client will be able to follow a drawn line to the marker on their client version of the map and also change depending on the means of transportation the client uses.

What is complex in this project: The main complexity that seems to me would be building the whole eco system you would need for this project, because not only do you need build the client side and server side and make sure all the components are connecting together to make this eco system work which is whole mission on its own, but you also have to accommodate both IOS and Android operating systems, which can literally double the workload as you would have to test the client and host on both systems and also make sure the app can adapt to both the IOS and Android systems, and while getting permissions to build and deploy your app on Android is simple enough, with IOS it becomes a whole different story because you need to get permissions to even build apps on apple devices.

What technical architecture was used: The architecture used is three-tier architecture as it can be split into 3 parts, the mobile app which the client, the 1st Droplet that has the server functions required for the app to function and the 2nd Droplet which contains the data store to provide routing data.

Project 2

Title: MindGame: A Machine learning Framework for Electronic Games

Student: Simon O'Neill

Description: As the title might suggest this is Machine learning algorithm that can be used to complete arbitrary goals in popular electronic games such as Snake, Pac-Man and Space Invaders, these goals are confined to the games so they mostly consist of reaching high scores and completing objectives more quickly and efficiently.

What is complex in this project: What seems the most complex to me in the project is getting the games connected to the AI and passing data between them both. This can be challenging because each game has a different interface requirement which means the frameworks needs to be able to understand the different source codes of each game well enough to ensure it complies to the frameworks rules to be able to achieve its goals.

What technical architecture was used: The learning algorithm contains the decider and game manager which are sending data between each other to get the inputs and outputs, base type of the input and outputs and also strings values of the inputs and outputs.

2.6 Conclusions

This concludes our research sections and should show how some of the following coming up sections were decided based on our research. It explored existing solutions that gave us a basis as to what components will be required for our project, such as picking the model type, in our case we will be using a CNN, the different libraries that will be required, such as RDKit and OpenBabel for cheminformatics or TensorFlow and PyTorch for building and training the neural networks, and were also able to find out what software we can use to put those libraries and packages together in environments so would be able to control the versions of each of these and all so they were all always readily available for the model to use as these environments can be imported and exported into different IDE's and software's.

3. Experiment Design

3.1 Introduction

This section we will be going over how the following system architecture was designed and made, which will show the process how each of the different systems interact with one another in the architecture and what information we obtained from the research to influence the design of the overall system and deployment of the model.

3.2 Software Methodology

Software methodology plans the framework that is implemented for the development lifecycle of a specific software systems. Applying the software methodology makes sure that the developer or team of developers follows a set of stages to provide an efficient and fast project delivery. These methodologies are more useful for large scale development but can also be used in any project so that it would maintain quality for every stage of the development process.

For the project the agile methodology was chosen at the started as it seemed like it was most suited, but as the project went on different issues and strategies had to be taken it became obvious that this development process would be better suited for the agile scrum strategy, this is because the development cycle repeated every two weeks as every week a new feature be introduced or some system change had to be implemented to utilize the new features on a weekly basis.

So once the ML model had it data ready to go and was trained this already put us off track for the agile plan as how fast model could be trained dependent on many variables, but once it was trained we could start incorporating different features in 1-2 week periods as the rest of the components didn't depends on how fast our GPU or CPU could perform. My supervisor acted as scrum master as this is only single person project, so every week I would discuss what in plan to implement and then showcase it every 1-2 weeks.



Figure 14 Scrum diagram

3.3 Overview of System

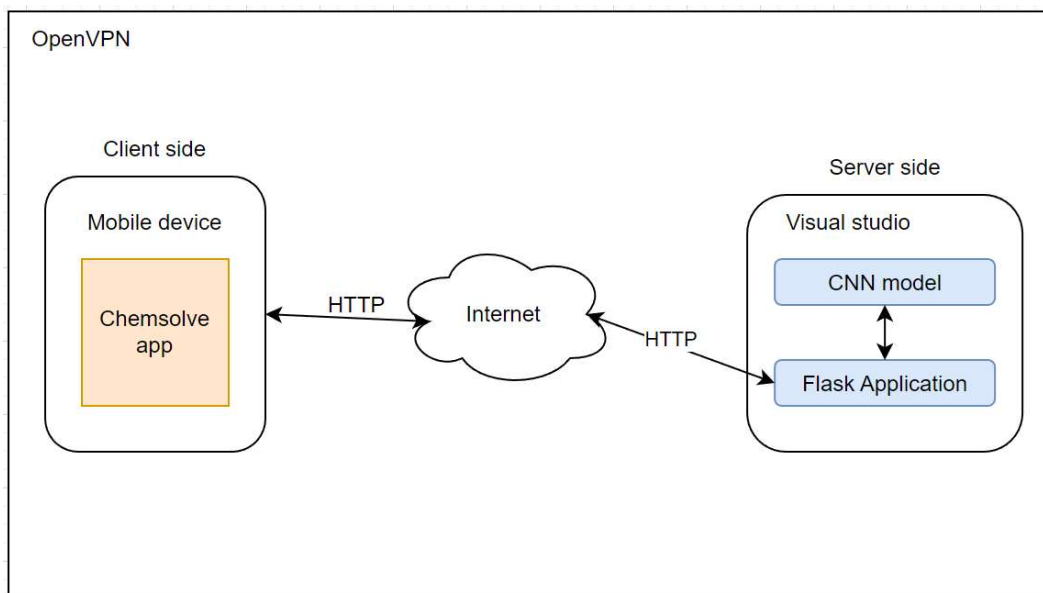


Figure 15 Project System Architecture

The system architecture can be divided into two distinct components:

- First have the mobile application (ChemSolve) which will be our client.
- The second component is the server side, this is the flask application interacting with our ML model.

The mobile application has two UI functions for the user, one for the user to be able to see and interact with the information gotten from the server and the other for interacting with the server to get that information.

The server side is a flask web app running from visual studio and whenever the code is running it starts the server making the app port visible for the mobile app to connect to. This flask app is responsible hosting the machine learning model, to where it can send images from the mobile app and get the information from that image to be sent to the mobile app.

While those two components can connect if the port forwarding is activated properly the connection isn't very secure nor those it offer any high encryption when connecting or transferring data, and since the port forwarding isn't working properly on my router I could only use it to connect over my local WiFi, to get around this I used OpenVPN to make tunnelling protocols to create a secure connection.

3.4 Software test plan

There are many ways we could test this software such as:

Acceptance testing where we check the application as a whole to see if it is indeed working as it should.

Unit testing which is testing the smallest testable components in the systems so see its doing what it is meant to.

Regression testing which is making test functions for each function to see if it would break with any new functions.

Integration testing to see if all the software components fit well together.

Stress testing which is a type of test to see if the system will break with enough stress.

There were many test I could have done and implemented, but looking at the scope of the project and considering it's a full stack development project with CNN models and hints of 3D rendering, scripting and development, I just couldn't fit in any of the tests in time for the deadline of this report as there were many, many components and pieces I had to figure and overcome just to get all the systems to talk to each other and get it fully functional.

3.5 Front-end

The plan for the front-end is an android mobile app that will allow the user to interact with the server with its interface and also to be able to display the data retrieved from the server.

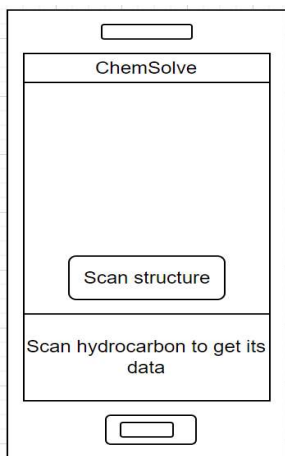


Figure 16 basic app UI

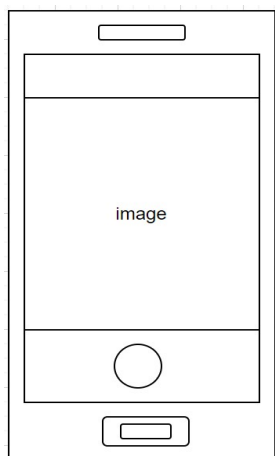


Figure 17 App using phone cam

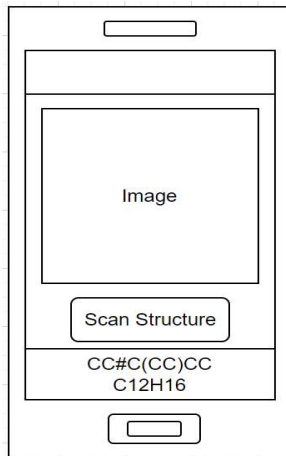


Figure 18 Server returning SMILES

The above wireframes is how I think the system for the frontend will look like, if not like this it will still have a similar functionality to this. The objective of the front is to send the server data and

display the retrieved data and to do this we need to pass the server an image, to do this my idea is it use a button that when pressed let's the user use the phone camera to take a picture. Once the picture is captured it is sent to the server.

Once the image is processed and its data is retrieved, the image taken will be displayed on the screen with the data of that image displayed under button in its own section. So the main functionality of the front-end is to be able to take scan an image, send that image to the server and retrieve its data to be displayed to the user.

Additionally, to this we also want to display the structure as the a 3D structure on the interface this should look something like this:

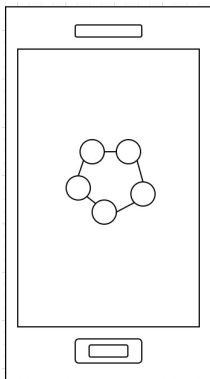


Figure 19 3D structure being displayed

This will most likely be done by having the 3D cords retrieved from the server and getting a game or rendering engine to read the cords and display the 3D structure on the android app.

Considering we are trying it display 3D objects in the android app I think the best choice for this will be Unity to render and display the object and have it be inside the android app in android studio as you can integrate Unity into android studio very easily.

3.6 Middle-Tier

The middle-tier will be the model we will be using to classify the image and outputting the smiles data and also adding additional functions to be able to find more information form that Smiles string.

```
This message will be only logged once.  
2023-03-27 03:52:50.963725: I tensorflow/stream_executor/cuda/cuda_blas.cc:1614] Te  
C#CC(=C)CC1CC=CC1(C)C  
InChI=1S/C12H16/c1-5-10(2)9-11-7-6-8-12(11,3)4/h1,6,8,11H,2,7,9H2,3-4H3/t11-/m1/s1  
The compound name for the given smiles string is 4-isopropyl-3-methyl-2-hexene.  
chemical formula: C12H16
```

Figure 20 Getting SMILES string and additional information from the SMILES

Before we can make the model operable we need to get appropriate datasets for said model, there could be multiple ways to do this, you could look through databanks for images of hydrocarbons and

label each one of them, you could try search different websites that may already contain labelled datasets but the problem with both these options is that it could take a long time collecting all this data for the model to be somewhat accurate as we would need at least 100k images. So an interesting way to overcome this is actually an example that one of the researched project showed us, instead of them collecting thousands image they actually generated the multitude of datasets and got the scripts to label them as well.

Once the data has been generated the model can then be built and generated, and the basic functionality that we need from the AI is to be able to take an image as an input, process that image and tell us the respective SMILES string of the image, the reason why we only need the SMILES string is because that format can actually give us a lot of information about the molecule from the SMILES string such as the chemical formula, InChi string, compound name and 3D coords.

3.7 Back-End

The backend will consist of 3 different components, the 1st being server-side as we need to deploy the model and make it visible over the network for it to be interacted with, 2nd is the client-side which will be on the mobile app this will connect to server and send/retrieve data from the server and 3rd we need to make the server-side visible over all networks and to do this we need to either port forward, host it on a domain, use sockets or create tunnels between the two components.

The server-side component can be done by hosting the model in a custom web framework such as in flask or fastAPI, these will essentially turn the model into web app that will be visible over the local network for the client side to connect.

For the client-side to connect and send information to the server we need to find something to make requests and handle responses in a simple manner, so that it would send the image as a request for it to be processed and handle the response from the server.

The final part is that we need to make server be available over any network and not just locally, I don't think the domain route will be suitable as it will come at a cost to have that domain and also the port forwarding can be insecure and can even not work depending on your router company and if your public IP isn't static that can also cause problems, so for this solution I think using something like OpenVPN or Ngrok would be suitable to make protocol tunnels between the two components.

3.8 Conclusions

This concludes our experimental design stage and hopefully this gives an idea to the reader how the design decisions were made and that the features we require from the front-end, middle-tier and back-end have ended up shaping our designs for each of our components and systems such as the front-end needs to be able to send image so either one way or another the user has to be able to access the camera from their mobile device using the app or how even though the back-end works all well perfectly in a local network we need make it also work for mobile data and other Wi-Fi networks, which can be very messy and requires security and encryption as well which is why one of

the most simplest ways is to use protocol tunnels as they can make connection server to client secure and simple.

4. Experiment Development

4.1. Introduction

In this section we will be going over how the software was developed for each of these components from the generating data scrips to the scripts used for Unity to make 3D structures, this will give an insight for the choices made throughout the software portion of the project and hopefully give an idea why the project was designed this way. I will also be showing how to set up the project and datasets for the code.

4.2. Software Development

4.2.1 Project Set-up

The following essential hardware and software were used for this project:

- PC
- Android Mobile Device

- Android Studio
- Visual studio
- Unity
- Blender
- OpenVPN
- Flask
- Postmaster
- Anaconda
- Jupyter Notebook
- C++
- Java
- Python 3.6
- Tensorflow 2.10.1
- OkHttpClient
- OpenCV
- Cuda 11
- CuDNN 8

The PC and android mobile device were required for the development and testing of the server and application. The CNN model was also trained and tested on the PC, now this where this project might not work on your laptop depending on what specs you have, for example I wanted to see the comparison of training just 50k data on my CPU vs my GPU and the results were a lot more wider than I thought.

To give you an idea of how wide the margin actually is, a 50k dataset on the PC's CPU takes 19000 seconds which equates to approximately 5 hours, now the model is trained over 30 epochs, so $5 \times 30 = 150$ which if divided by 24 hours (full day) equates to 6.25 days per 50k data which there are 3 of, so to train the model on all 3 would take approximately 18 days. You might be thinking 18 days isn't too bad, but the real problem is the model is only at 86% accuracy at 100k dataset so that 18 days just turned exponentially bigger, now if you want 98+ % accuracy your going to have to train it on 500k datasets which as you can imagine would probably takes months which I don't think any mid-spec laptop could handle. Now to give you a comparison for the GPU on 50k dataset, the time took 829 seconds which is 0.2 hours, $0.2 \times 30 = 6$ hours, $6 \times 3 = 18$ hours, so while it might take the CPU 18 days the GPU gets it done in 18 hours, and to give in perspective for 500k on GPU takes 7000 seconds so 2 hours, $2 \times 30 = 60$ hours, which is 2.5 days and $2.5 \times 3 = 7.5$ days. So, as you can see even the 500k takes less time to train on the GPU than the 50k on CPU also note so that there isn't any confusion the k stands for thousand, so 500k is 500,000 images, and also while it's training your PC or laptop will have limited use as the model will be using a lot of resources to train it. This also means that a lot of that time you won't even get to work on your other components or get to test them as you want as it takes more time and also can be at risk of crashing and even stopping the model from training.

model_info_50kClean.txt - Notepad	model_infoGPU_50kClean.txt - Notepad
File Edit Format View Help --Clean RDKit 50K-- epoch 1/30 - training = 19494s - loss = 0.8046 - perplexity = 2.2781 - lr = 0.0001 Eval: - BLEU-4 = 50.27 - EM = 0.00 - Edit = 64.30 - perplexity = -1.85	File Edit Format View Help --Clean-RDKit-50K-- epoch 1/30 - training = 829s - loss = 0.8653 - perplexity = 2.4095 - lr = 0.0001 Eval: - BLEU-4 = 41.88 - EM = 0.00 - Edit = 62.19 - perplexity = -1.93

Figure 21 50k dataset on CPU

Figure 22 50k dataset on GPU

```
- Training: - 7716s - loss: 0.3382 - perplexity: 1.4673 - lr: 0.0001  
- Evaluating...Loaded 40548 formulas from results/data_clean-RDKit-500K.json/  
formulas_val/ref.txt  
Loaded 40548 formulas from results/data_clean-RDKit-500K.json/formulas_val/hy  
p_0.txt
```

Figure 23 500k 1st epoch on GPU

A high spec laptop might be able to do the job, but I'm not sure because the PC has 6 cooling fans and AIO pump keeping down the temperature and even with that it still gets very hot, so I don't know if a laptop would be able to not overheat. To give an idea of the PC's specs ill list them below:

- RTX 3080
- 32GB RAM
- AMD Ryzen 7 5700G
- ROG STRIX B550-F

Android studio IDE was installed and used to make the android app, the language used to do this was Java, it was also used to make various android virtual devices for application testing.

Visual Studio was used to develop and run the flask web app, so the server is technically running from visual studio. You can also use custom environments to accommodate our ML model and we will also be using Python and C++ to make the scripts for the project.

Unity will be used to display our 3D structure in android studio, its relatively easy to set up a scene with lighting and camera, it can use C++ scripts to generate the structure.

Blender will be used to make some of the more complicated assets as its creation system is much easier to use than Unity and we can also just import the blender assets into Unity.

We need OpenVPN to make tunnels between the server and mobile app so that our devices could access the server over any network.

The flask web framework is going to be used to make a very simple web app to host the ML model so that it was visible to our mobile app and to allow connections to the server.

Postmaster is used solely to test the model and server, as the OpenVPN, android app and port forwarding can take some time to set up and might not be able to test the ML model like we would want, we can instead use postmaster to post images to the server to test its responses.

Anaconda and Jupyter Notebook are used together as you can use Jupyter Notebook in anaconda. But mainly Anaconda is used to make the custom environments and to use Jupyter Notebook which is used to generate the data, train the model and evaluate it too all in python.

OpenCV is used in generating the data, as we need it for image processing techniques such as image filtering, rotating or distortions.

TensorFlow is the machine learning library that we going to use to build our model and its individual layers, this needs to be as of time of writing this version 2.10.1.

Cuda 11 and CudNN 8 work together to give our training, testing and functionality of our model a significant boost by using our GPU instead of our CPU which is what those packages are. Now those specific software version are for the RTX 3080 paired with TensorFlow 2.10.1, if you have a different GPU then you would need to look what versions would match your GPU and what TensorFlow versions support your chosen Cuda and CudNN versions.

OkHttpClient is library we will be using in android studio for our app so that it would be able to make network requests to the server and handle its responses.

These are all the main and essential software's, libraries, hardware and languages to make a complete copy of this project set up.

4.2.2 Generating Training Data

The data we generate can be divide into 3 different types, our clean images, our hand-drawn images and our synthetic images, the reason for these different types is so that our model could recognise images even with a lot of background noise and obstructions, so no matter what's in the image the classifier will still be able to pick out the structure and classify it.

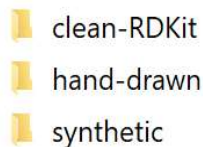


Figure 24 Data folder

Now each folder has its own scripts and folders that are used to generate the data used, and since each dataset is different each folder has its own different components so we will be going over all 3 folders.

- clean-RDKit data

To start we will be looking at the at the clean data folder:

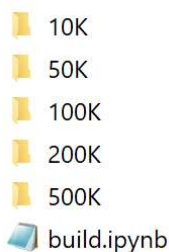


Figure 25 clean-RDKit folder

As we can see we have datasets for 10k, 50k, 100k, 200k and 500k, this indicates how large the datasets are so for example 500k has 500,000 images. In each of these folders we have text files, and these text files contain numerous hydrocarbons in the form of SMILES strings which are accompanied with index files to number them. The reason why we have numerous test files is because we need to split the datasets in to test, train and validation datasets.

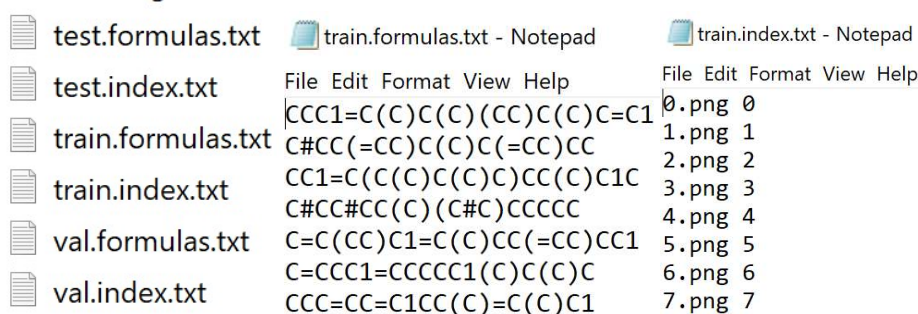


Figure 26 text files

Figure 27 Hydrocarbons as SMILES

Figure 28 Index for each SMILE

That doesn't mean that each text file has the amount of SMILES string equivalent to folder number e.g. the 500k gets split up for testing, training and validation, so all together they make up 500k or 200k or however big the dataset is. Once we have all those SMILES and index numbers saved, we can make a script to start generating it, the clean data script is simple but gets more complicated as we get to the other datasets.

First we need to make a function to read in the text file and take each string from that text file to then be returned so that it could be turned into an image.

```
In [2]: def get_smiles(filename):
        with open(filename) as f:
            lines = f.readlines()

            smiles = [s.split()[0] for s in lines]

        return smiles
```

Figure 29 function to read SMILES text file

That function is used to read the text files and split up each individual SMILES string, next we need to make a function to turn that SMILES into an image to do this we can make a function using RDKit the cheminformatic library we have referenced many times before.

```
In [3]: def smiles_to_rdkit(s, i, img_dir):  
  
        m = Chem.MolFromSmiles(s)  
        AllChem.Compute2DCoords(m)  
  
        Draw.MolToFile(m, "{}/{}.svg".format(img_dir,i), wedgeBonds=False)  
        svg = svg2rlg("{}/{}.svg".format(img_dir,i))  
        renderPM.drawToFile(svg, "{}/{}.png".format(img_dir,i), fmt="PNG")  
        os.system("rm {}/{}.svg".format(img_dir,i))
```

Figure 30 function to generate images

To explain a bit about the above function it basically takes 3 variables *s* for SMILES, *i* for index and *img_dir* is the image folder its going into. Once the function has these 3 arguments it can start generating images by translating SMILES into Mols and turning them Mols into 2D coordinates using RDKit library. Once turned into 2D cords, RDKit has a function to be able to draw them 2D cords on image files such as png and svg and able them with their respective index number and then added them to their respective directories.

This should create 3 such directories as mentioned before from the generation script:

- test_images
- train_images
- val_images

Figure 31 image directories

Also note that in the above code we have “wedgeBonds” set to false, this is because we want to specify not to include any wedge bonds when RDKit draws the structure.

Here is an example of what a wedge bond molecule looks likes:

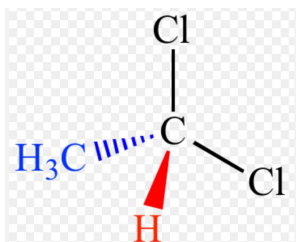


Figure 32 Molecule w/ wedge bonds

The wedge bonds are the blue and red bonds in figure 41.

Instead, we want the clean data molecules to look like this:

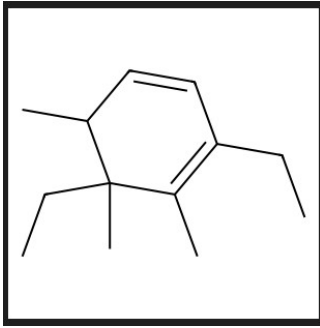


Figure 33 Hydrocarbon generated w/ our script

- Synthetic data

The next data to be generated is the synthetic data, we do this one second because the data generated from this will be used in the hand-drawn data.

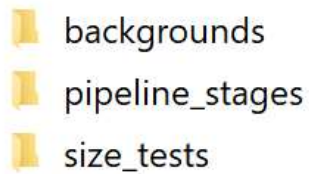


Figure 34 Synthetic data folder

The backgrounds folder is as it says a folder full of backgrounds, this is to add these backgrounds to every image so that our model could become accustomed to different background noise as its being trained.

Here are some of the backgrounds being used to give you an idea what data to collect:

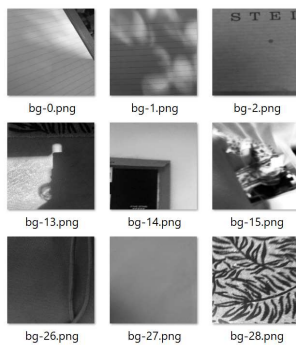


Figure 35 Background pictures

Now with the background images collected we can start generating the pipeline data. The reason its called pipeline data is because instead of the datasets being divided into size they are all a size of 200k and divided by how many modifications they have as seen in image below.

- rdkitp
- rdkitp-aug
- rdkitp-aug-bkg
- rdkitp-aug-bkg-deg

Figure 36 Pipeline datasets

As can be seen from figure 36 there are 4 different stages:

-rdkitp: this is data that has been slightly modified such as making the lines wider and crooked.

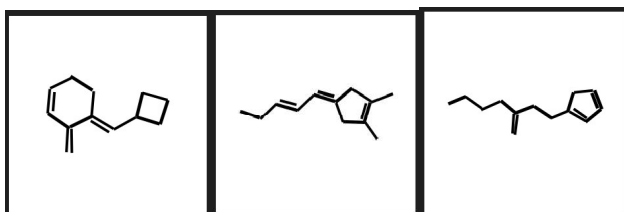


Figure 37 examples of rdkitp images

-rdkit-aug: this data was also modified like the rdkitp but on top of that we use a more augmentation such as making the lines blurry and distorted and moving it to different positions on the image.

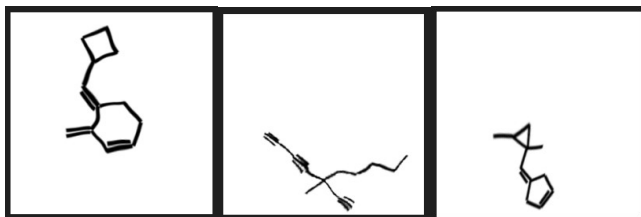


Figure 38 examples of rdkitp-aug images

-rdkit-aug-bkg: this data has all the modifications as the ones above but also includes the various backgrounds in our background folder.

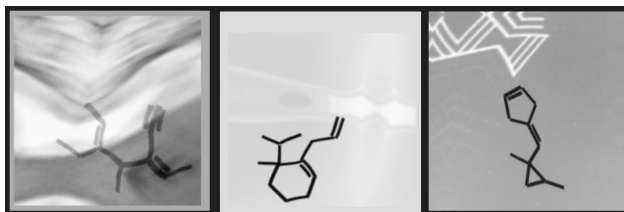


Figure 39 examples of rdkitp-aug-bkg images

-rdkit-aug-bkg-deg: this also contains all the modifications as stated above but also has its own degradation function which changes the structure such as making some lines wavy, changing the colour of the structure and changing the angles of the bonds.

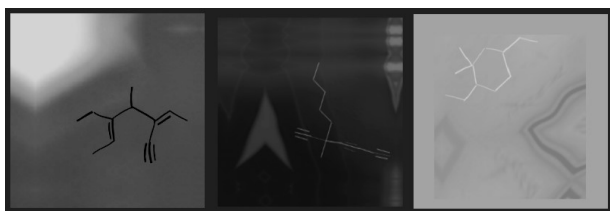


Figure 40 examples of rdkitp-aug-bkg-deg images

That's the 4 examples of image sets that we have for the pipeline stages and we can see by each stage the images get more and more degraded and augmented. The generation is similar to the clean data only difference is that they use external class functions such as augmentation.py and degradation.py which contain a bunch of other functions such as rotation, elastic transformation, blur, dilate etc.

We also have size_tests datasets, these are split up into sizes of 50k, 100k, 200k and 500k. These datasets are similar to the pipeline ones as they also use the degrade and augment functions and have their backgrounds replaced, but main difference is that this data will be used to be trained with the hand-drawn data.

- Hand-drawn data

The next dataset to be generated is the hand-drawn dataset, this is to recognise hand-drawn hydrocarbons of different hand writing styles to minimize any errors for all types of styles. Now this folder is set up bit differently.

This how it looks like inside the hand-drawn folder:

- 📁 hand-drawn-full
- 📁 hand-drawn-training
- 📁 hand-drawn-val
- 📁 test-set

Figure 41 Hand-drawn folder

In figure 41 you can see we have 4 different folders, that includes the training, validation and test datasets, in which we can already see it differs from the structure of the folder in figure 34. Now the structure of this folder is little weird as we have 2 validation datasets, one is in the hand-drawn-val folder and one is in the hand-drawn-training folder.

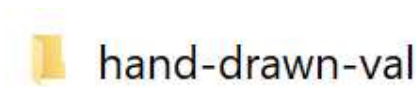


Figure 42 validation set 1



Figure 43 validation set 2

The reason for this will be explained in more detail later in the training section but for now to give an idea basically we have 2 validation sets for hand-drawn images, 1 is to be used with the hand-drawn training data and one is to be used with the synthetic data.

```
"dir_images_train": "Z:/FYP/data/synthetic/size_tests/500K/train_images/",
"dir_images_val": "Z:/FYP/data/hand-drawn/hand-drawn-val/val_images/",
"dir_images_test": "Z:/FYP/data/hand-drawn/test-set/test_images/",
```

Figure 44 JSON file paths for training synthetic data

```
"dir_images_train": "Z:/FYP/data/hand-drawn/hand-drawn-training/training-sets/0_100/train_images/",
"dir_images_val": "Z:/FYP/data/hand-drawn/hand-drawn-training/validation-set/val_images/",
"dir_images_test": "Z:/FYP/data/hand-drawn/test-set/test_images/",
```

Figure 45 JSON file path for training hand-drawn data

As we can see from figure 44, I have a JSON file using synthetic images as the training data and the hand-drawn-val as the validation data, and then figure 45 is using the hand-drawn-training data as the data set and the validation-set as the validation data.

You might have noticed that the hand-drawn folder also has another folder in figure 41 called hand-drawn-full this a folder full of hand drawn hydrocarbons, we have this because RDKit can't exactly turn SMILES strings into hand drawn structures, luckily since its hand drawn it wont take us long to look for this data as we can just draw the individual hydrocarbons got from the clean data.

The hand-drawn training set also has augmentation and degradation functions applied to it but is split up into ratios of modifications instead of dataset sizes.

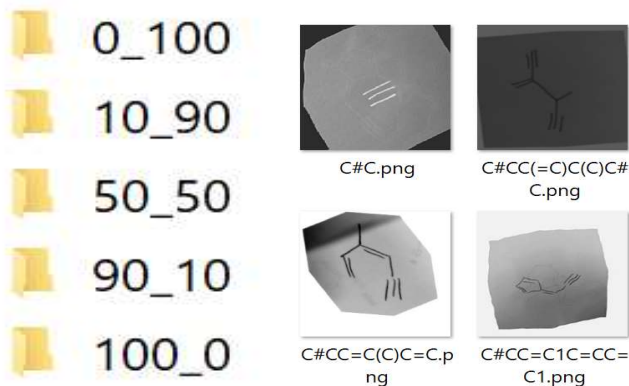


Figure 46 hand drawn ratio folders

Figure 47 hand-drawn training set images

4.2.3 Machine learning Model

Although it would be cool to make our own CNN model for this project we unfortunately don't have the time or resources, as this project is looking at the capabilities of AI's can have with mobile devices and cheminformatics and not into the specifics of designing the CNN model ourselves as to do that we would have to look at lots of different model's to figure out how they work and then work on one ourselves which would have taken up majority of this project. Even just setting up the models like shown in other projects can be a challenge in itself which will be discussed below.

The model I have chosen for this project is one from ChemPixCH by the [mtzgroup](#)[7], this is a CNN type model, which based on our research of different model types I think is most suited for this project.

The model consist of several layers which can be grouped into convolutional layers, pooling layers and fully connected layers.

The convolutional layers are used to extract features from the input image, by applying filters to the image and identifying patterns and features.

The pooling layers basically aggregate the features extracted by the convolutional layers by downscaling it and taking the maximum or average value to reduce parameters in the model.

The connected layers take the outputs of the convolutional layers and the pooling layers and uses these outputs to classify the chemical structure based on the features extracted.

If you want more information about this specific CNN model you check out the “Recognizing Hand-drawn Hydrocarbon Structures with Neural Networks”[8] link and read all about it in more detail.

The main challenge with using this model was getting it set up, since it was made about 2 years ago according to the ChemPixCH [7] github a lot of versions of libraries it uses are out of date.

```
name: im2smiles
channels:
  - rdkit
dependencies:
  - python=2.7
  - rdkit=2018.09.3.0
  - opencv
  - scikit-learn=0.20
  - numpy
  - make
  - pip
  - pip:
    - tensorflow==1.1.0
    - scipy==1.2.2
    - nltk==3.4.1
    - networkx==2.2
    - tensorboard==1.9.0
    - distance==0.1.3
    - click==6.7
    - multiprocessing
    - matplotlib==2.2
    - scikit-image<0.15
    - pillow
    - logging
    - six
    - svglib
```

Figure 47 ChemPixCH environment file

As we can see this environment uses python 2.7 which isn't supported by many IDE's, software's and libraries. This is big problem for us because if we want to train the model using our GPU we need the CUDA libraries, but to use the CUDA libraries we need higher version of python and TensorFlow. This is where Anaconda comes in very handy, because while we can use visual studios or other python IDE's to run this set up it will be very difficult getting the correct libraries and packages to match the correct versions and could get very messy.

So, what we can do instead is make a custom environment with a new version of python such as 3.6 which was used for this setup. To do this we can use anacondas PowerShell to make a new environment and specifically for this for this environment we want to use Python version 3.6 and

TensorFlow 2.10.1. Then once that environment is made, we can activate in anaconda to set it as the active environment.

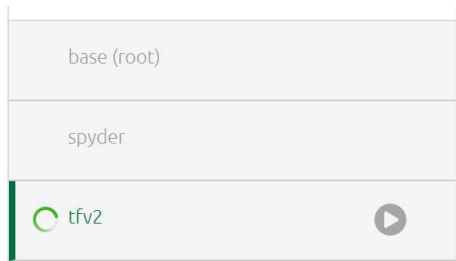


Figure 48 activating our custom env in anaconda

Once this environment is active, we can start to set up all the classes and functions needed for the model to work, to this I just copied the code from each class in the github and pasted it into Jupiter notebook. Most of the classes can just be copied and pasted directly as .py files but to be able to run the training and evaluation scripts we need to turn them into ipynb files. We don't need to turn these back into .py files as they won't be used in Visual Studio as all the training will be done in jupyter notebook.

The whole reason we are running the training scripts in jupyter notebook instead of visual studio is because we need to be able to easily install the other packages and libraries required for the model and this is easier to do in Anaconda rather than visual studio. So whenever we run the training script it will go through all the classes and functions and tell us what libraries are missing and we can then install them on our environment using pip or conda but since we are using Python 3.6 it will install us the version of each of these libraries and packages that match our version of Python and TensorFlow which effectively makes our custom environment. This might take a few tries to get right as it took me, but it is possible to get it working it's just very nuanced and can take some time downloading and installing all the packages and libraries.

Once the custom environment is working the other problem we have is that the code was written on python 2.7 and TensorFlow 1.1.0 which has a lot of outdated code, especially TensorFlow since we are using essentially version 2. What I did for this is first try get it working on TensorFlow 1.1.0 to see if it works, once working we need to switch to TensorFlow 2.10.1 (the reason for the switch is so that it was CUDA 11 compatible so we could use our GPU). This is where the custom envs are useful because we can just switch between them so we can have one for TensorFlow 1.1.0 and one for TensorFlow 2.10.1. Now once I used TensorFlow 2.10.1 I got a lot of errors as a lot of the code used was deprecated in version 2 of TensorFlow. This is the part of the project where it became very daunting for me as after seeing all these errors, I wasn't sure if this model was viable because on one hand if I use TensorFlow 1.1.0 it will take forever to train the model and it won't be possible to get the other parts done or have it ready with full accuracy but on the other hand TensorFlow 2.10.1 breaks the code and gives me loads of errors about TensorFlow libraries being deprecated which doesn't lead me anywhere it as I don't know if it's possible for the model to work without these pieces of code and libraries.

I was close to going out and looking for a completely different model but the problem with this is that it would take a long time trying to get the appropriate data and setup for the model to work, so before I took that route I saw something on the TensorFlow website that caught my eye and this was

the ability for TensorFlow to automatically turn version 1 code into version 2 code. This could be done with the following line:

```
tf_upgrade_v2 - infile foo.py - outfile foo-upgraded.py
```

Figure 49 code to upgrade tf code

The code takes an infile foo.py, in our case we can change foo with our .py file names and the outfile is the upgraded code which we can name whatever. To my surprise this worked effectively well and actually got rid of a lot of our errors without us having to change the logic of the model.

Some other minor challenges were that once we turned the Python version to 3.6 is all broke how some of the code worked as it was written on Python 2.7 so I had to manually change some of that code which wasn't too difficult.

Another difficulty was that when training the model you need reference is JSON files that are in the configs file, this might not be obvious as there are a lot of folders and files and it seems like you can just reference your data set directly but that is not the case, the JSON files actually reference the datasets and we need to change them to match our datasets.

```
{
  "export_name": "data.json",

  "dir_images_train": "Z:/FYP/data/clean-RDKit/500K/train_images/",
  "dir_images_val": "Z:/FYP/data/clean-RDKit/500K/val_images/",
  "dir_images_test": "Z:/FYP/data/clean-RDKit/500K/test_images/",

  "path_index_train": "Z:/FYP/data/clean-RDKit/500K/train.index.txt",
  "path_index_val": "Z:/FYP/data/clean-RDKit/500K/val.index.txt",
  "path_index_test": "Z:/FYP/data/clean-RDKit/500K/test.index.txt",

  "path_formulas_train": "Z:/FYP/data/clean-RDKit/500K/train.formulas.txt",
  "path_formulas_val": "Z:/FYP/data/clean-RDKit/500K/val.formulas.txt",
  "path_formulas_test": "Z:/FYP/data/clean-RDKit/500K/test.formulas.txt",

  "bucket_train": false,
  "bucket_val": false,
  "bucket_test": false,

  "max_iter": null,
  "max_length_formula": 50,

  "buckets": [
    [240, 100], [320, 80], [400, 80], [400, 100], [480, 80], [480, 100],
    [560, 80], [560, 100], [640, 80], [640, 100], [720, 80], [720, 100],
    [720, 120], [720, 200], [800, 100], [800, 320], [1000, 200],
    [1000, 400], [1200, 200], [1600, 200], [1600, 1600]
  ]
}
```

Figure 50 changed JSON training file

Once this was all set up and done, we are ready to train and test the CNN model to be used for our project.

4.3 Front-End Development

The front end was developed in android studio with the help of Unity and Blender. Android studio was used to develop the code for the base of the app and give simple functions to the user such as giving the app permissions to use the camera so that the user can take a picture to be sent to the server.

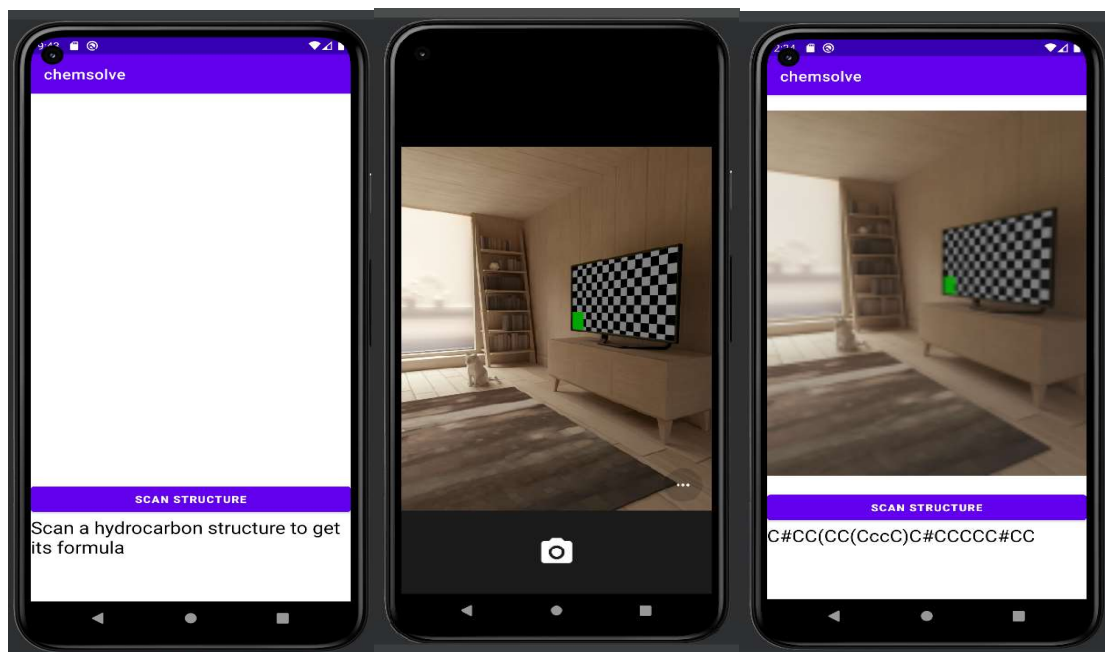


Figure 51 basic app UI

Figure 52 App using phone camera

Figure 53 Server returning SMILES

The above images show how the UI is meant to function, these are images all taken from android studios emulator function which lets you emulate any android device on any SDK this specific emulated device is a Pixel 5 with API 29 and android version 10.0, this is one reason why android studios was chosen as the specific software to write the android mobile app as it gives lots of possibilities to test the app on different versions of android phones and SDK versions.

First the user presses the scan structure button which will allow the user to scan any digital or hand drawn hydrocarbon structure. This is done by when the button is pressed and assuming you gave the app camera permissions for your phone, it will open a camera instance on the mobile app and allow you take a picture, the picture then gets sent to the server to where it gets processed by the model and a prediction is made of the structure is, then once all the server side data has stopped processing it sends that data back to the app. The picture taken is also saved and then displayed in an image view container with the relative data displayed underneath the picture (chemical name, formula etc will also be included but this early prototype that only displays SMILES for now)

The second part of this mobile app is to display the 3D structure obtained from the SMILES notation, this is done in the Unity instance inside the mobile app.

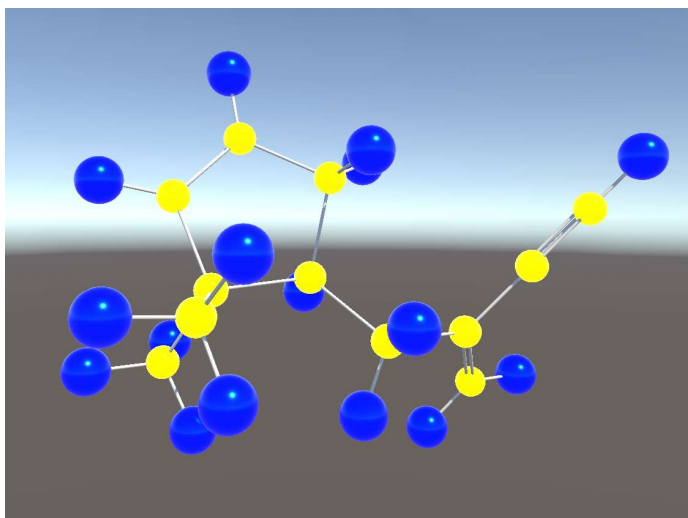


Figure 54 Unity 3D structure

Now when the image was sent to the server and the SMILES was retrieved from the image we could do multiple things with that SMILES string such as find the chemical formula and name, but another function it has is that it can be transformed into set of instructions in the form of SDF file.

```

|
|      RDKit      3D
|
|      0 0 0 0 0 0 0 0 0 0999 V3000
M  V30 BEGIN CTAB
M  V30 COUNTS 22 21 0 0 0
M  V30 BEGIN ATOM
M  V30 1 C 5.574471 1.422292 -0.125935 0
M  V30 2 C 4.166055 1.099617 0.096700 0
M  V30 3 C 3.009518 0.834489 0.278563 0
M  V30 4 C 1.526268 0.494429 0.511554 0
M  V30 5 C 0.369260 0.229237 0.693670 0

```

Figure 55 molecule SDF file

In the above SDF file we can skip until the line 'BEGIN ATOM', don't pay attention to the M V30 and look at what comes after, the 1 stands for the 1st atom, 2 is 2nd atom etc, the letter after the number stand for the type of atom e.g. C stand for Carbon, H for hydrogen etc then 3 numbers after is where the atom is positioned in the space.

For unity to understand this I had to write my own custom script to read these values and assign atom prefabs to each line, so the first prefab will be a carbon atom e.g. the yellow sphere and will be labelled as atom 1 carbon at the positions 5.574, 1.422 and -0.1259. The reason why we want to label atoms with their assigned number is because we later need to read in bonds.

```

M  V30 BEGIN BOND
M  V30 1 1 1 2
M  V30 2 3 2 3
M  V30 3 1 3 4
M  V30 4 3 4 5

```

Figure 56 bond instructions from same SDF file

So now the above instructions can be read as the first number stands for which bond it is e.g. 1 is the 1st bond 2 is the 2nd bond, then the second number stands for what bond type it is so 1 is a single bond, 2 is a double bond and 3 is triple bond, the following number after that is what atoms is the bond between. So for the first line 1 1 1 2 means that's the first bond is a single bond between atoms 1 and 2, the second line 2 3 2 3 stands for the 2nd bond is a triple bond between atoms 2 and 3 etc.

Writing the script was quite difficult as one of the main challenges to solve was that SDF shifts positions so for example if the molecule is more than 10 atoms long it will automatically shift all the lines by 1 forward since its going from single digit to double digit e.g. from 9 to 10 adds +1 positional value to each values after that and also if one of the coordinate had a - in front of it would also add another +1 position to each value after the cord with the minus in front of it. This essentially meant that the script would be reading the wrong values unless I had it somehow dynamically change the position it takes the value from depending on pervious conditions.

```
if (LineNumber >= 10 && lineNumber <= 40)
{
    shift1 = 12;
    shift2 = 21;
    shift3 = 30;

    string check = line.Substring(12, 1).Trim();
    Debug.Log("check if " + check);

    if (check == "-")
    {
        w = 10;
        shift2 = 22;
        shift3 = 31;

        string check2 = line.Substring(22, 1).Trim();
        Debug.Log("check if " + check2);
        if (check2 == "-")
        {
            shift3 = 32;
            string check3 = line.Substring(32, 1).Trim();
            if (check3 != "-")
            {
                w = 9;
            }
        }
    }
    else if (check2 != "-")
    {
        w = 9;
    }
}
```

Figure 57 dynamically checking when to shift positions

As can be seen from above code the way I got around this was using if statements. First I counted and specified for what values the script should record the cords from, so the first was to start reading from 11 spaces, then 20 for second cord and 29 for the third. The w stands for the width of the cord which also had to be dynamically adjusted. Then if the cord was between 10 and 40 add +1 to all the spaces so now cord 1 started on space 12, 2 on space 22 and 3 on space 30. Then we need to check if the first cord has a '-' is yes add +1 to spaces for cord 2 if no the stay the same and same for cord 3 which had to be shifted twice if both cords had '-' and once if only one had '-'. This is essentially how I got around this challenge by updating the shifts depending on what atom number it is and if it has a '-' or not. The same procedure was done for the bonds worked surprisingly well with no errors, as if it did have an error then the structure would not be able to be generated.

This SDF file when made is sent to the app where its read in by the Unity script. The mobile app then makes a Unity instance to where it reads the instructions assigns premade prefabs to the positions of the atoms (prefabs are just premade objects that can be used from anywhere later on, in our case the blue and yellow spheres are premade objects I made earlier to be assigned as the atoms) and also makes the bonds to each atom including double and triple bonds. This full structure is then

displayed on the screen to where the user can interact with the structure by rotating it with their fingers.

The reason Unity was used for this part instead of some other 3D viewer is because Unity integrates into android apps and android studio very seamlessly and easily and as such it's easy enough to make it work inside a mobile app.

4.4 Middle-Tier Development

Once the image has been sent from the app to the server, the image is then parsed onto to the model where it starts up the model and takes the image as an input to be classified.

```
Z:/FYP/im2smilestf-lite/results/clean-RDKit-100K/vocab.json
Z:/FYP/im2smilestf-lite/results/clean-RDKit-100K/model.json
Building model...
```

Figure 58 Model building after been given an input.

The above image shows the model starting up when the image was sent to it, as can be seen its using the weights made for the 100k clean data as it's the only weights available as of writing this section.

Then once the image has been through the layers and has been declassified by the model it will output a SMILES string as shown below.

```
This message will be only logged once.
2023-03-27 03:52:50.963725: I tensorflow/stream_executor/cuda/cuda_blas.cc:1614] Te
C#CC(=C)CC1CC=CC1(C)C
InChI=1S/C12H16/c1-5-10(2)9-11-7-6-8-12(11,3)4/h1,6,8,11H,2,7,9H2,3-4H3/t11-/m1/s1
The compound name for the given smiles string is 4-isopropyl-3-methyl-2-hexene.
chemical formula: C12H16
```

Figure 59 CNN model outputting information about the image.

That smiles string is also then converted into chemical name, inChi formula, chemical formula and SDF file which can be seen above, this is then sent onto the mobile app. Note that this is also subject to change because as the chemical name and everything else on it is relevant and important information there might be more uses for the SMILES string we can add later.

Another thing you might not know and a challenge I had to get around is that we actually use 2 machine learning models the first being the CNN model for the SMILES prediction and the second being chatGPT, now I know that chatGPT also can read images and could probably classify the images to SMILES but when developing this chatGPT 4 had not been released and it didn't have image classification functions.

The reason chatGPT is used is because as I've mentioned above in the research section is that for us to get the IUPAC name of an hydrocarbon we need to follow a specific set of rules and depending on how you interpret the molecule and rules you can get different names for the SMILES, these names aren't necessarily wrong as 1 molecule can be interpreted in multiple ways and as such can have

multiple names, this makes it a perfect candidate to solve using another AI and I was originally going to look for something that can do this but since time was limited and chatGPT was very good at these type of things with a very seamless and easy implementation. All you needed was the openai library, a api key and a prompt to send to chatGPT which worked very well and fast.

```
openai.api_key = "sk-XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
prompt = f"Find using OPSIN what is the compound name for the following inchi string: {inchi_str}?"
response = openai.Completion.create(
    engine="text-davinci-002",
    prompt=prompt,
    max_tokens=1024,
    n=1,
    stop=None,
    temperature=0.5,
)
```

Figure 60 integrating chatGPT

4.5 Back-End Development

The back-end also consists of 2 different components, one is the flask code ran on visual studio to start the server and the second one is the OpenVPN connecting the server and the mobile app.

In visual studio we have a server script that basically turns the python script into web app, and in this web app the flask container imports classes from the model that are used to initiate the model, I basically have a class called img2smiles that was specifically made to be imported into a web framework which is deploying our model when the server is started up, so that whenever it receives an image it would start up the model and get it to classify the image.

```
* Serving Flask app 'server'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.123:5000
```

Figure 61 flask web app starting up

The image above shows what it looks like when the code is first started up and ran in visual studio, it starts the web app and makes it available to connect to for the mobile app. As can be seen this is only development server and I'm using this because we don't need a fully-fledged server as I'm only trying to show how useful and how AI's can be incorporated into chemistry in a fun and very useful manner, as such we can use a development server to showcase this since for this project I don't plan to deploy the app to the play store on android.

Another important thing to note in that image are all the connections its running, as we can see it says 'Running on all address (0.0.0.0)', this means that it will take a connection from any device that has reference to its addresses, and what I mean by references to its address is the two other lines 'Running on <http://127.0.0.1:5000>' and 'Running on <http://192.168.1.123:5000>', there is another address given by OpenVPN but we will get into that later. But what those two lines refer to is that when server starts up it has two IP's you can connect to on whatever local network the server is running from on port 5000 as seen by :5000. Now 127.0.0.1 is just the localhost address that I used for testing and 192.168.1.123 is the address used to test the port forwarding, but note these

addresses only work from the local network the server is running from so in my case that is my home WiFi.

Also another side note that the server needs to be running with the python environment that was used to train and test the model in Anaconda, this is why we use visual studio for running the flask web frame as visual studio can automatically detect environments made in anaconda and be used in its IDE for python scripts as well, which saves us the trouble of having to export the environment file and findings ways to import them into other IDE's or software the flask app might be running on.

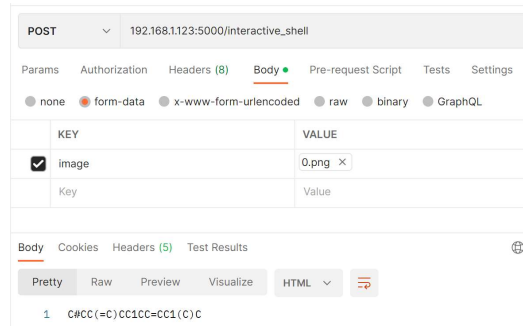


Figure 62 testing port forwarded address

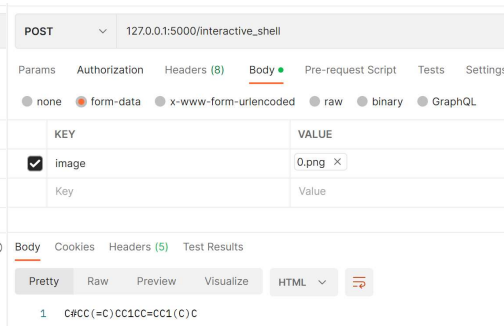


Figure 63 testing local address

I mentioned above that there was a third address we can use that does not show up in the console when the server is started up, this is because the two above addresses are only ever going to be used when you are connected to the same network as the server, so in other words if you try use 3G/4G or different WiFi then those address will not work and as mentioned before this is because even though my port forward might be set up correctly my actual WiFi company haven't assigned me the correct settings to make it work, also since those address don't cant be accessed by outside networks they don't need encryption as they will only work from the network that my server is connected to.

YOUR PRIVATE IP
172.27.232.17

Figure 64 address given by OpenVPN

To over come this we use the above address given to us by OpenVPN, for this to work the address must be referenced in your mobile app and also the OpenVPN must be turned on your phone and whatever device the server is running from, in my case my PC.

```
.url("http://172.27.232.16:5000/interactive_shell")
```

Figure 65 adding address to mobile app

Once turned on and the address has been referenced by the mobile app, you can start the server, and this will create a protocol tunnel from the mobile app to the server. OpenVPN also encrypts all

the data transferred from device to the server and vice versa which takes care of any security issues with might have encountered using the port forwarding.

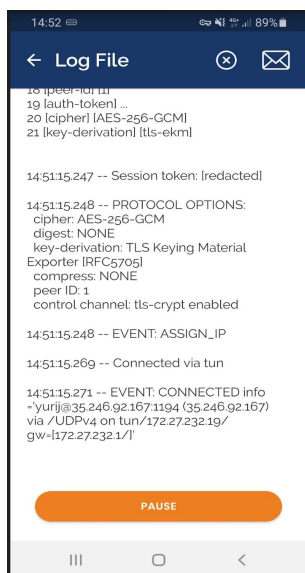


Figure 66 OpenVPN starting session on mobile device

The above image shows what the Log file of the OpenVPN shows when you are connected, as can be seen it makes a session token and uses a cipher to encrypt the data.

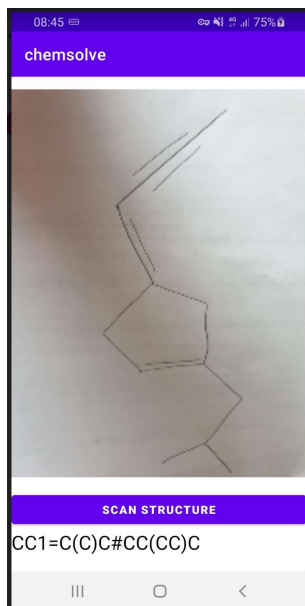


Figure 67 App working using 4G

Once you're all set up and good to go you can start to scan images with the app on any network whether it be on a different WiFi network or 3G/4G mobile network without having to worry of having your data, server or devices compromised.

The reason OpenVPN was used over Ngrok is that Ngrok has a session time of 2 hours so not very good for testing and showcasing, while OpenVPN is active until turned off and also while providing high encryption it also gives us a Log file of the connection events so we can spot any errors.

4.6. Conclusions

This concludes our experimental development, we go through how to set up the project with the various hardware, software, tools and libraries and why we need these and also show in detail how each section of the project was solved from generating the different types of training data to showing how we made protocol tunnels between the server-side and client-side and each step of the way showing the reasoning and decisions made while developing the software, such as using Anaconda to develop custom environment so that we could upgrade the model and train it on our GPU for maximum efficiency.

5. Testing and Evaluation

5.1. Introduction

This is the testing and evaluation section of the project, here we want to test and see how accurate our model is on different data sets to see if there is an actual improvement between it being trained on 100k data and 500k, to test the actual application to check the difference between its actual behaviour and expected behaviour and also have a user to test it in the form of questionnaires or surveys where users with different technical expertise try to use the app without any assistance to let us know how easy or confusing the UI is to use and if any problems popped up throughout the testing stages. Also as mentioned in section 3 I couldn't set up a proper project system test plan as I didn't have enough time nor resources for this very tight schedule to be able to implement these specific test classes and functions.

5.2. System Testing

5.2.1 Model accuracy

The first part of the system I want to test is how accurate the model actually is and if there is an actual improvement in accuracy using 500k instead of 100k data, specified by this graph on the ChemPixCH [7] which shows how accurate the model gets compared to images in dataset.

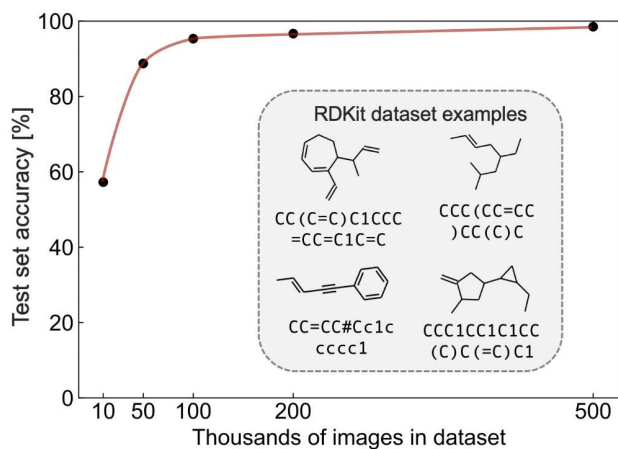


Figure 68 model accuracy graph

As can be seen from graph above the more images the model is trained on, the more precise the model is. This can also be seen from the model.log where it gives our various model scores:

Epoch 30/30

- Training: - 1381s - loss: 0.0054 - perplexity: 1.0054 - lr: 0.0001
- Eval: BLEU-4 98.46 - EM 92.76 - Edit 98.11 - perplexity -1.02
- New best score (-1.02)!
- Saved model in results/clean-RDKit-100K/model.weights/

Figure 69 model evaluation results for 100k

Epoch 29/30

- Training: - 7837s - loss: 0.0020 - perplexity: 1.0020 - lr: 0.0001
- Eval: BLEU-4 99.55 - EM 97.95 - Edit 99.44 - perplexity -1.00
- New best score (-1.00)!
- Saved model in results/data_clean-RDKit-500K.json/model.weights/

Figure 70 model evaluation results for 500k

As we can see the results for 100k gives a high score of perplexity of 1.02 and 500k gives a score 1.00 and what this technically means is that the closer the model is to 1 then the more accurate it is as 1 means that the model is predicting an exact match for each test and validation image.

To do this accuracy test I will be taking images from the 200k validation set and predicting the SMILES for these images using the model being trained on 100k weights and 500k weights, the reason we are taking images from 200k dataset is because the model hasn't been trained on 200k making it perfect dataset to test on.

Actual SMILES	Predicted SMILES	True or False
<chem>C#CC(C)(CC=CC=C)C(=C)C</chem>	<chem>C#CC(C)(CC=CC=C)C(=C)C</chem>	
<chem>C=CCC(=C)C1CCC(=C)C1(C)C</chem>	<chem>C=CCC(=C)C1CCC(=C)C1(C)C</chem>	
<chem>C=C1CC(CCCC)=CC=C1C</chem>	<chem>C=C1CC(CCCC)=CC=C1C</chem>	
<chem>C=C(C#CCC)CC(CC)CC</chem>	<chem>C=C(C#CCC)CC(CC)CC</chem>	
<chem>C#CC(CC)C(C)C=C(C)C</chem>	<chem>C#CC(CC)C(C)C=C(C)C</chem>	
<chem>CC#CCC(C)(CCC)CCC</chem>	<chem>CCC#CCC(C)(CCC)CCC</chem>	
<chem>C#CCC(C)(C)C(C)=C1C=CCC1</chem>	<chem>C#CCC(C)(C)C(C)=C1C=CCC1</chem>	
<chem>C=C1CCC(C)(C=CC=CC)C1=C</chem>	<chem>C=C1CC(C)(C=CC=CC)C(=C)C1</chem>	
<chem>C=C(C)C(C=CC)CC1CC1C</chem>	<chem>C=C(C)C(C=CC)CC1CC1C</chem>	
<chem>C#CCC1(C)CCCCC1C#C</chem>	<chem>C#CCC1CCCCC1C#CC</chem>	
<chem>C#CC1(CCC)CC(C)C=C1C=C</chem>	<chem>C#CC1(CCC)CC(C)C=C1C=C</chem>	
<chem>CC(C)C1CC1CC1(C)C(C)C1C</chem>	<chem>CC(C)CC1CC1CC11C(C)C</chem>	

<chem>C#CC=C1CC=CC=CC(C)(C)C1</chem>	<chem>C#CC=C1CC=CC=CC(C)(C)C1</chem>	
<chem>CC1C(C)C1C#CCC1CC1</chem>	<chem>CC(C#CCC1CC1)C1C(C)C1C</chem>	
<chem>C=CC(C)=CC1CCC1(C)CC</chem>	<chem>C=CC(C)=CC1CCC1(C)CCC</chem>	

So for the above 15 images we got a total of 10 out of 15 correct which is 66.67% accuracy for the 100k dataset weights.

Actual SMILES	Predicted SMILES	True or False
<chem>C#CC(C)(CC=CC=C)C(=C)C</chem>	<chem>C#CC(C)(CC=CC=C)C(=C)C</chem>	
<chem>C=CCC(=C)C1CCC(=C)C1(C)C</chem>	<chem>C=CC(=C)CCC(C)C(=C)CC1C</chem>	
<chem>C=C1CC(CCCC)=CC=C1C</chem>	<chem>C=C1CC(CCCC)=CC=C1C</chem>	
<chem>C=C(C#CCC)CC(CC)CC</chem>	<chem>C=C(C#CCC)CC(CC)CC</chem>	
<chem>C#CC(CC)C(C)C=C(C)C</chem>	<chem>C#CC(CC)C(C)C=C(C)C</chem>	
<chem>CC#CCC(C)(CCC)CCC</chem>	<chem>CC#CCC(C)(CCC)CCC</chem>	
<chem>C#CCC(C)(C)C(C)=C1C=CCC1</chem>	<chem>C#CCC(C)(C)C(C)=C1C=CCC1</chem>	
<chem>C=C1CCC(C)(C=CC=CC)C1=C</chem>	<chem>C=C1CCC(C)(C=CC=CC)C1=C</chem>	
<chem>C=C(C)C(C=CC)CC1CC1C</chem>	<chem>C=C(C)C(C=CC)CC1CC1C</chem>	
<chem>C#CCC1(C)CCCCC1C#C</chem>	<chem>C#CCC1(C)CCCCC1C#C</chem>	
<chem>C#CC1(CCC)CC(C)C=C1C=C</chem>	<chem>C#CC1(CCC)CC(C)C=C1C=C</chem>	
<chem>CC(C)C1CC1CC1(C)C(C)C1C</chem>	<chem>CC(C)C1CC1CC1(C)C(C)C1C</chem>	
<chem>C#CC=C1CC=CC=CC(C)(C)C1</chem>	<chem>C#CC=C1CC=CC=CC(C)(C)C1</chem>	
<chem>CC1C(C)C1C#CCC1CC1</chem>	<chem>CC1C(C)C1C#CCC1CC1</chem>	
<chem>C=CC(C)=CC1CCC1(C)CC</chem>	<chem>C=CC(C)=CC1CCC1(C)C</chem>	

As can be seen from this the 500k was more accurate than the 100k but got 1 wrong where 100k got right, this can be fixed by loading in multiple models using the 100k and 500k and whichever gets the prediction more accurate, then that's the one it should output. The accuracy approximately is 86.67% which is like a 20% increase compared to 100k so if tested it on more 200k data that accuracy % might have been higher.

5.2.2 Black Box testing

To test the overall application and the different systems in place black box testing was performed to test the expected behaviours. The reason we are doing black box testing is because automated testing wouldn't really be able to show and justify the test results as we need to see with our own eyes how the system error checks the images you send it.

Test	Expected result	Pass	Fail	Comments
Try to take picture without structure in the image.	The model should still predict a SMILES string but the SMILES string should be rendered invalid as it isn't a real structure.			If the algorithm doesn't detect an accurate string it outputs to the user the issue.
Try to use app with mobile data.	The app should work on 3G/4G and give a response from the server.			For this to work the phone and server need to have OpenVPN active
Try scan hand drawn image.	The model should return a appropriate response to the image.			For this to work we need to have our model trained on hand-drawn data
Try scan an image that is digital or has lots of background noise.	The model should return a appropriate response to the image.			We need the synthetic data for this work as digital images contain lots of noise if your taking a picture of another screen.
Try see what 3D structure the app generates on invalid SMILES.	The 3D structure should be empty and should display that it isn't valid SMILES.			If the SMILES is invalid then no SDF file is made for Unity to read.
See if the app generates any additional info for invalid SMILES.	The system displays the "SMILES is invalid" and nothing else.			If the SMILES is invalid then it skips the part where it gets the information about the SMILES.
See what output we get for structures that aren't hydrocarbons.	The system should display invalid SMILES			This is because our model uses a specific vocab and can only register Carbon

				and Hydrogen atoms.
Try scan an image with 2 structures.	We get an invalid SMILES string as the model scans both structures and combines the SMILES together making it invalid			
Try the same structure in brightly lit environment and then dimly lit environment.	The system should have the same response for both images.			We have multitudes of datasets that deal with different brightness so this shouldn't cause a problem.

5.3 System Evaluation

From the testing the model on the 200k data we can see that the accuracy does indeed increase the more data you train the model on, the only interesting discovery is that the 100k actually predicted one right compared to the 500k that got it wrong, to address this I'm using multiple weights from different datasets and comparing the SMILES they predict to get which one is the most accurate so even if 500k got it wrong 100k still has a chance to get a better answer that will be displayed.

We also went over all the error checking of the app and systems in place and while there probably is a way to break these systems to get them to do what they aren't meant to, I feel like a lot of these vulnerabilities have been already covered by the error checks in place.

5.4 Questionnaires and Interviews Evaluation

So, for this evaluation process questionnaires and interviews are used for users to fill out and answer so we can find out what they think is good and bad about the app and UI etc, but this wasn't a really viable option for this project as it has a lot of nuisances such as first of all the system isn't made for large scale use, just to demo as it uses OpenVPN to make tunnels and both the user and server need to have this VPN activated with the appropriate profile. Secondly, I'm not putting it up on the app store for people to directly to download and use as stated in the first point it isn't properly hosted and OpenVPN is only used for demo purposes so if I did want someone to test it, they would have to come to my home PC and get it downloaded straight from android studios. And finally, this is an android app which already decreases the amount of people I can ask to test this app as majority in Ireland use apple anyways.

But if I were to use a questionnaire to evaluate this it would look something like this:

1. On a scale of 1-10 please specify how useful do you think this app would be to you or other people.

2. If you were to use this app in, what situations do you think you are mostly likely to use it.
3. Do you think the information you gain from this app is adequate? And if so or not why?
4. Is there any other information you would like to see about the hydrocarbon?
5. On a scale of 1-10 please specify how easy it is to use the UI? Give a reasoning for your rating.
6. If you could add another feature to this app, what would it be.
7. Do you prefer the interactive 3D UI or a simpler one with buttons and images.
8. If you were back in school or college how likely are you to use this app?
9. Since the app is a mobile app and not a web app, does this instil a pull towards using it over web apps?
10. Is there any other software's or apps you use that are similar in functionality to this one.

5.4. Conclusions

In conclusion with the limited resources and time that we had we have made cohesive evaluation of the system and model that we are using, I would have liked to test the app a bit more including some additional features, but we just don't have the time and I would also have liked to get some more user input and feedback to the application so I could see what else we could have potentially added or fixed to fit users' needs more specifically. But overall to show that the AI's are indeed very powerful companion tools for cheminformatics in mobile apps was very successful as the core of this project depends on the CNN being able to work as its meant to.

6. Conclusions and Future Work

6.1. Introduction

Now coming to the end this will be our final section of our report where we will talk about how overall the whole project went and if we achieved to prove the challenges we wanted to undertake with this project, there have been many challenges and problems that we had to overcome and face and we will now see if it has all come to fruition in the final end result, although there were many parts we had to skip and cut out of the final plan because of lack of time and resource, I still think the end result will be satisfactory and anything else that I did want to add to this overall system will also be discussed in this section.

6.2. Conclusions

We set out on a journey to find out why exactly AI's have taken over our current market, from the apps and virtual assistants that we use to the self-driving cars and ChatGPT's. I wanted to specifically find out how it can help out in chemistry as that is a particular difficult area of science and our daily lives that we encounter often. When going into this project I didn't even know if any of this was possible or how far could the AI even take me, well with the development of Chemsolve we can see exactly how useful and how much AI's can actually do to help speed up and learn different chemical processes to make our lives easier and maybe even help us learn and understand a little bit more about chemistry. Just from doing this one project I can think of another 6 development projects that I could incorporate AI's into chemistry even further, all this has shown me exactly why AI's are so widely used as it's a piece of technology that can exponentially improve our lives and lead us to new discoveries, especially when it works.

Chemsolve had the purpose of being able to read hand drawn or digital images made by the user and giving us as much information as possible for these images to make learning or translating these chemical structures that much easier. I don't think right now there are many useful applications on the app store for something like this so more of these type of apps that could make chemistry easier and more fun could always bring in more people to enjoy it. Chemsolve is currently limited to hydrocarbons as if it were to try and identify anything else it would take a very long time to implement, but that doesn't mean it's not possible.

To develop a project of this magnitude we need a variety of tools and different technologies that were researched to decide which could have been of use to this project, and since the project comprises of both client and server side making it a full stack development app plus 2 whole machine learning models, we needed these efficient tools and technologies or else the app would have never gotten finished.

For the client side we made an interactable UI that can send and retrieve data from the models which allows us to send images and get information back from them, this was all mostly done in Android studio as it gave us a simple and easy setup to be able to get this functionality to work while also giving us a many devices and SDKs we could test and use. Android studio was also a main choice because of its integration with Unity which allows us to even turn that 2D drawing into a 3D structure for users to see and interact with.

For the server side we used a simple flask web framework to hold the models and host them so that it could accept connections from any device connected to it. The reason we used flask was because it was very easy to use and I already had previous experience with it, and in this project where time is of the essence, I need to use software's and libraries that didn't have much setup. Although I would have liked to serialise the model and put in the mobile app to skip all the sever side stuff I don't think that would have been fully possible and might have even reduced computational ability. I even would have liked to host the app on actual domain to put on the app store but with all our constraints flask and OpenVPN was the only option to get a demo version out there.

To be able to safely handle the user data HTTP communication we use OpenVPN to create protocol tunnels between the app and server, it also provides encryption on any data sent or received which makes it that much more the viable option to use. Now while this method is a bit inefficient, it is free, fast and secure, so for demo version I think it's perfect.

While developing the project I did learn about a variety of different technologies not being taught to us in school and one of main objectives was to get better understanding of machine learning models which with this project I think I definitely achieved, from getting and generating the data to setting up and training the model there was a lot I learnt about it, and not even just CNN models but a few different ones as well.

The finished product is reliable and does indeed perform well in most cases, however I would have wanted to deploy the model a different way for more of commercial and public use, because as I am the only one using it I don't know what could use work on how the UI could be improved more specifically to suit other users, I would also want a working version on IOS and for it be able to classify more than just hydrocarbons.

6.3. Future Work

Throughout the project there were many ideas and functions I would have liked to implement and develop for this app but because of time restriction they have had to been dropped, here is a list of future work that could be done.

1. Since the model only works on hydrocarbons, that makes it very limited to only being able to scan hydrocarbons and although there are hundreds of hydrocarbons I think it would be useful for the model to be able to read more different types of structures. To do this we could either increase the vocab and training sets of the CNN model or make sperate models for different types of structures.
2. To find the IUPAC name I was originally going to make my own model to predict this, but since we had limited time, I had to fall back on ChatGPT, and while chat has been very useful it can also just give you a bunch of different names for the same molecule, so I would also consider making my own model for that part too.
3. I also wanted to add a feature where the user can add multiple structures to one display, so they could scan 2 different hydrocarbons and have them both displayed on the screen.
4. With the above feature I wanted to take it 1 step further and show how the 2 structures would interact with each other and balance each other out, for this part I would also need another machine learning model to predict the reactions between the two molecules and the products of the two while also balancing the whole equation.

5. I mentioned as well in section one that one very hard part about chemistry was the maths especially when it included ratios and so on, I think this is perfect problem to solve using code.
6. I would also like to deploy the models in a more proper manner so that it would be publicly available to everyone to use, test and see how cool chemistry actually is. So instead of having protocol tunnels we can host it properly on a domain.

Bibliography

1. Chemink by Xuguodong [1] "Handwritting recognition for some chemistry-related characters. Use yolov3 to cut strokes.": <https://github.com/xuguodong1999/chemink>
2. MolAR by the [mtzgroup](#)[2] Sukolsak Sakshuwong, Hayley Weir, Umberto Raucci, and Todd J. Martínez , "[Bringing chemical structures to life with augmented reality, machine learning, and quantum chemistry](#)", J. Chem. Phys. 156, 204801 (2022) : <https://github.com/mtzgroup/molar>
3. Im2latex by [Guillaume Genthial](#) [3]. "Image to LaTeX (Seq2seq + Attention with Beam Search) – Tensorflow" : <https://github.com/guillaumegenthial/im2latex>
4. DECIMER by Kohulan[4] Rajan, K., Zielesny, A. & Steinbeck, C. DECIMER 1.0: deep learning for chemical image recognition using transformers. J Cheminform 13, 61 (2021). <https://doi.org/10.1186/s13321-021-00538-8> : https://github.com/Kohulan/DECIMER-Image_Transformer
5. Android and IOS market shares [5], [6] <https://www.bankmycell.com/blog/android-vs-apple-market-share/>
6. ChemPixCH [7] <https://github.com/mtzgroup/ChemPixCH>
7. Recognizing Hand-drawn Hydrocarbon Structures with Neural Networks [8] : <https://www.rsc.org/suppdata/d1/sc/d1sc02957f/d1sc02957f1.pdf>